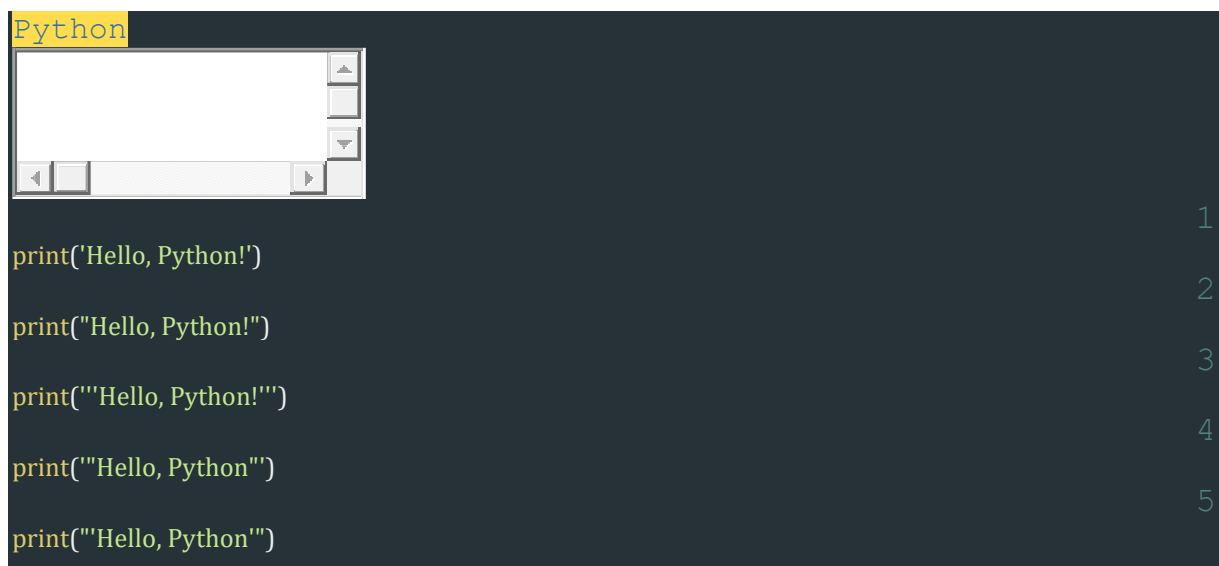


Python Basic Coding Interview Questions

1) Write the output for the below Hello, Python commands.



```
Python
print('Hello, Python!')
print("Hello, Python!")
print("""Hello, Python!""")
print("""Hello, Python""")
print("""Hello, Python""")
```

Output

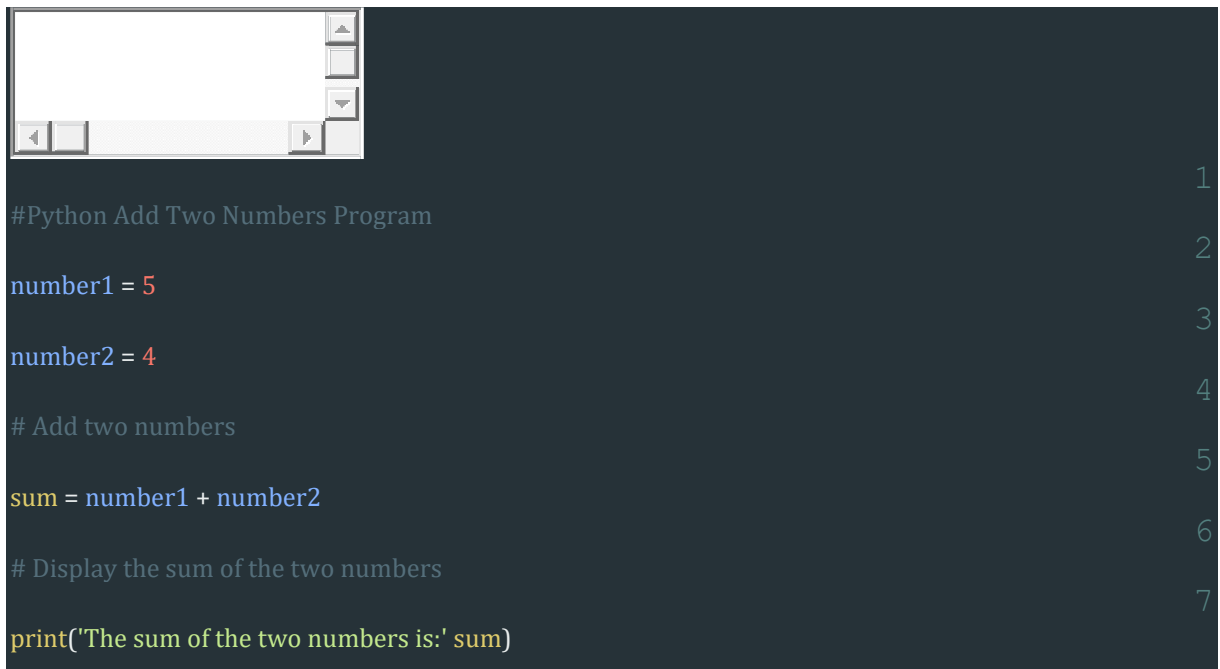
Hello, Python!
Hello, Python!
Hello, Python!
"Hello, Python!"
'Hello, Python!'

Explanation: In Python programming, a string can be enclosed inside single quotes, double quotes, or triple quotes.

2) What is the output of the Python add two numbers program?

Python Add Two Numbers Program





```
#Python Add Two Numbers Program

number1 = 5

number2 = 4

# Add two numbers

sum = number1 + number2

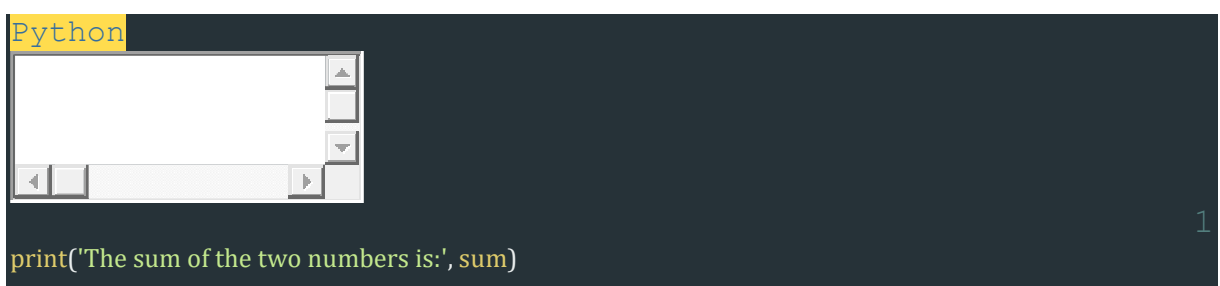
# Display the sum of the two numbers

print('The sum of the two numbers is:' sum)
```

Output of the program:

File "", line 7
print('The sum of the two numbers is:' sum)
^
SyntaxError: invalid syntax

The above Python add two numbers program throws SyntaxError because, the comma(,) is missing in print statement. The below print statement will give you the sum of the two numbers.



```
Python

print('The sum of the two numbers is:', sum)
```

3) What is the output of the below sum of the two numbers Python program?



```
Python
```

```
num1 = input('Enter first number: ')
num2 = input('Enter second number: ')
sum = num1 + num2
print('The sum of the numbers is', sum)
```

1
2
3
4

The output of the sum of the two numbers program

Enter first number: 15
Enter second number: 10
The sum of the numbers is 1510

Python input() function always converts the user input into a string. Whether you enter an int or float value it will consider it as a string. We need to convert it into number using int() or float() functions. See the below example.

Read the input numbers from users

```
num1 = input('Enter the first number: ')
num2 = input('Enter the second number: ')
```

Converting and adding two numbers using int() & float() functions

```
sum = int(num1) + int(num2)
sum2 = float(num1) + float(num2)
```

Displaying the sum of two numbers

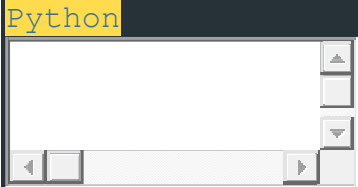
```
print('The sum of {0} and {1} is {2}'.format(num1, num2, sum))
print('The sum of {0} and {1} is {2}'.format(num1, num2, sum2))
```

The output of the sum of the two numbers program

Enter the first number: 15
Enter the second number: 10
The sum of 15 and 10 is 25
The sum of 15 and 10 is 25.0

4) Write a Python program to illustrate arithmetic operations (+,-,*,/)?

Here is the Python arithmetic operators program:

A small icon of a Python IDE window with a yellow title bar and a white text area, located at the top left of the code editor.

```
1
2 # Read the input numbers from users
3 num1 = input('Enter the first number: ')
4 num2 = input('Enter the second number: ')
5
6 # Converting and adding two numbers using int() & float() functions
7 sum = int(num1) + int(num2)
8
9 # Subtracting the two numbers
10 sub = int(num1) - int(num2)
11
12 # Multiplying two numbers
13 mul = float(num1) * float(num2)
14
15 #Dividing two numbers
16 div = float(num1) / float(num2)
17
18
19 # Displaying the results of arithmetic operations
20 print("The sum of {0} and {1} is {2}'.format(num1, num2, sum))
```

```
print("The subtration of {0} and {1} is {2}'.format(num1, num2, sub))
```

21

```
print("The multiplication of {0} and {1} is {2}'.format(num1, num2, mul))
```

22

```
print("The division of {0} and {1} is {2}'.format(num1, num2, div))
```

23

The output of the Python arithmetic operators program

Enter the first number: 25

Enter the second number: 10

The sum of 25 and 10 is 35

The subtraction of 25 and 10 is 15

The multiplication of 25 and 10 is 250.0

The division of 25 and 10 is 2.5

5) Python Program to Check if a Number is Odd or Even.

Python

```
#Python program to check if a number is odd or even
```

1

2

3

```
#To get the input from user
```

4

```
num1 = input("Enter a number: ")
```

5

6

```
#Checking whether the entered number is odd or even
```

7

```
if (int(num1) % 2) == 0:
```

8

```
    print("{0} is Even number".format(num1))
```

9

```
else:
```

10

```
    print("{0} is Odd number".format(num1))
```

11

The output of the Python odd or even program

Enter a number: 4

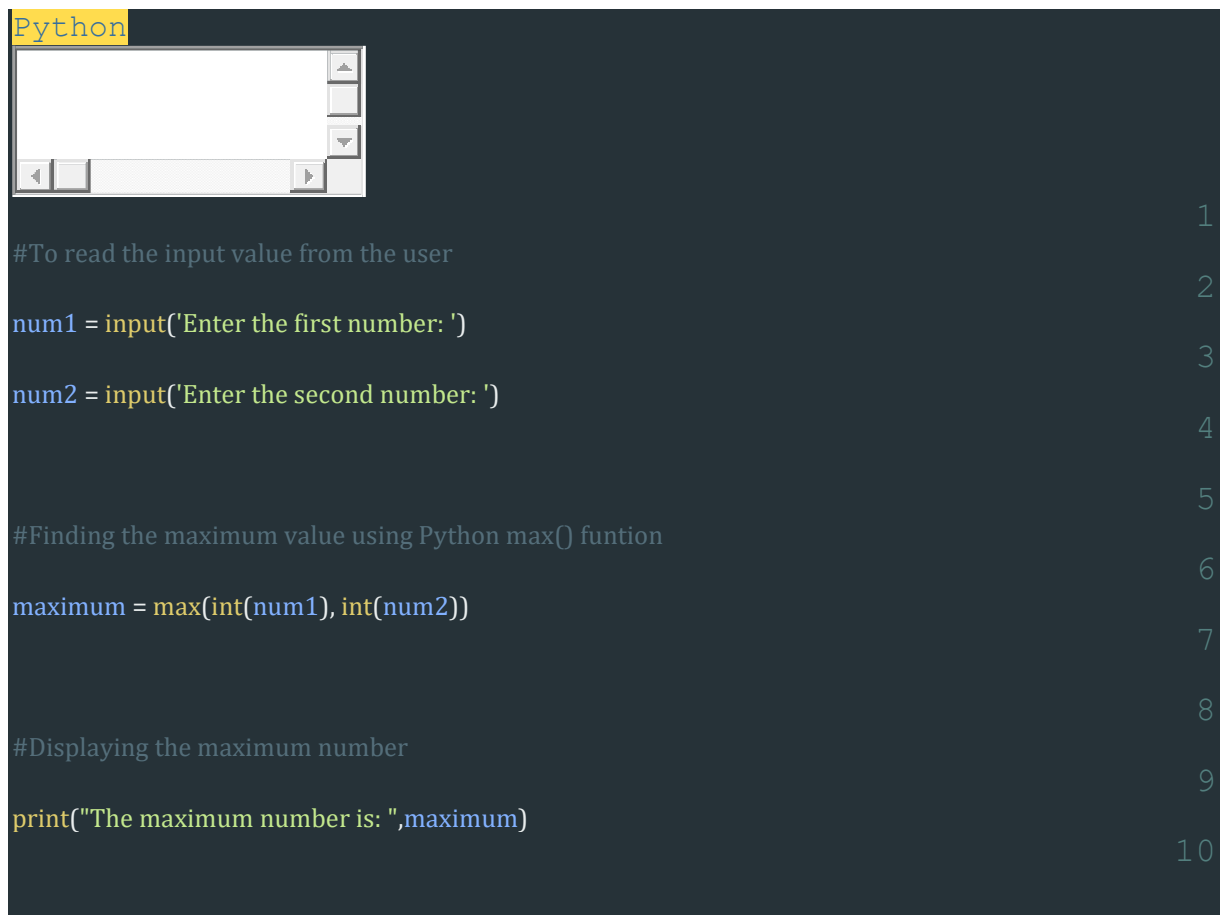
4 is Even number

Enter a number: 5

5 is Odd number

6) Write a Python program to find the maximum of two numbers?

Here is the Python program to find the maximum of two numbers:

A screenshot of a Python IDE with a dark background. The title bar of the editor window says "Python". The code is as follows:

```
#To read the input value from the user
num1 = input('Enter the first number: ')
num2 = input('Enter the second number: ')

#Finding the maximum value using Python max() funtion
maximum = max(int(num1), int(num2))

#Displaying the maximum number
print("The maximum number is: ",maximum)
```

Line numbers 1 through 10 are visible on the right side of the code editor.

The output of the Python program to find the maximum number

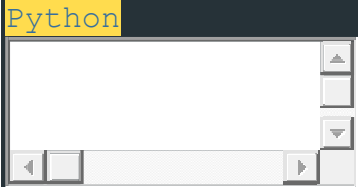
Enter the first number: 23

Enter the second number: 45

The maximum number is: 45

7) Write a Python program to check prime numbers.

Here is the Python program to check whether a number is prime or not:



```
1
2 # Function to check prime number
3
4 def PrimeChecking(num):
5
6     # Condition to check given number is more than 1
7
8     if num > 1:
9
10        # For look to iterate over the given number
11
12        for i in range(2, int(num/2) + 1):
13
14            # Condition to check if the given number is divisible
15
16            if (num % i) == 0:
17
18                #If divisible by any number it's not a prime number
19
20                print("The number ",num, "is not a prime number")
21
22                break
23
24        # Else print it as a prime number
25
26        else:
27
28            print("The number ",num, "is a prime number")
29
30        # If the given number is 1
31
32        else:
33
34            print("The number ",num, "is not a prime number")
35
36 # Input function to take the number from user
37
38 num = int(input("Enter a number to check prime or not: "))
39
40
```

```
# To print the result, whether a given number is prime or not
```

21

```
PrimeChecking(num)
```

#Output1 of the above Python program to check the prime number

Enter a number to check prime or not: 10

The number 10 is not a prime number

#Output2 of the above Python program to check the prime number

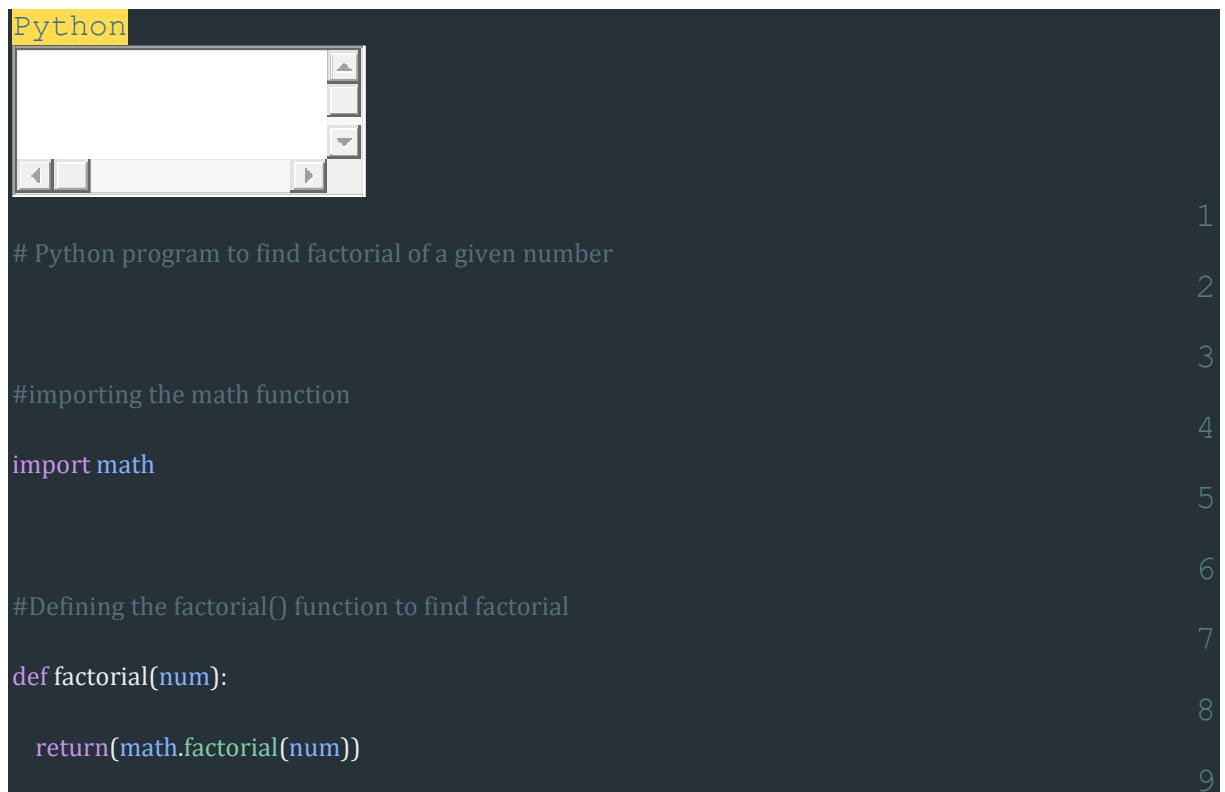
Enter a number to check prime or not: 37

The number 37 is a prime number

8) Write a Python factorial program without using if-else, for, and ternary operators.

Yes, we can write a Python program to find the factorial of a number using in-built Python functions.

math.factorial() function returns the factorial of a given number. Let's have a look at the Python factorial program using a math function:

A screenshot of a Python IDE with a dark background. The title bar of the editor window says "Python". The code is as follows:

```
# Python program to find factorial of a given number

#importing the math function
import math

#Defining the factorial() function to find factorial
def factorial(num):

    return(math.factorial(num))
```

Line numbers 1 through 9 are visible on the right side of the code editor.

```

# Input function to get the number from user

num = int(input('Please enter a number to find the factorial: '))

#Printing the factorial of the given number

print("The factorial of the given number", num, "is",

factorial(num))

```

Output1 of the Python factorial program

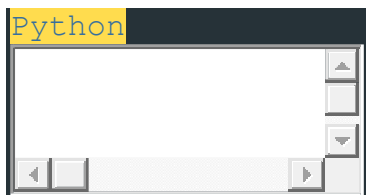
Please enter a number to find the factorial: 5
The factorial of the given number 5 is 120

The output2 of the Python factorial program

Pleas enter a number to find the factorial: 10
The factorial of the given number 10 is 3628800

9) Write a Python program to calculate the square root of a given number.

Here is the Python Program to calculate the square root:



```

# Input function to get the input from the user

n = float(input('Enter a number: '))

#Formula to calculate the square root of the number

n_sqrt = n ** 0.5

```

```
#Printing the calculated square root of the given number  
print('The square root of {0} is {1}'.format(n,n_sqrt))
```

Output1 of the Python program to find the square root

Enter a number: 2
The square root of 2.0 is 1.4142135623730951

Output2 of the Python program to find the square root

Enter a number: 4
The square root of 4.0 is 2.0

10) Write a Python program to calculate the area of a triangle.

To calculate the area of a triangle we need the values of side one, side two, side three, and the semi-perimeter. Let's have a look at the Python area of the triangle program:

```
Python  
# Python Program to find the area of a triangle  
  
# Get the 3 sides of a triangle from the user  
s1 = float(input('Enter first side value: '))  
s2 = float(input('Enter second side value:'))  
s3 = float(input('Enter third-side value:'))  
  
#Calculating the semi-perimeter of a triangle
```

```

9
10
11
12
13
14
15
sp = (s1 + s2 + s3) / 2

#Calculating the area of a triangle

area = (sp*(sp-s1)*(sp-s2)*(sp-s3)) ** 0.5

#Printing the area of the triangle

print("The area of the triangle is: ", area)

```

The output of the Python area of the triangle program

Enter first side value: 3
 Enter the second side value:4
 Enter third-side value:5
 The area of the triangle is: 8.48528137423857

11) Write a Python program to check Armstrong's number.

The armstrong number can be defined as n-digit numbers equal to the sum of the nth powers of their digits are called armstrong numbers.

Armstrong Number Example:

153 is a armstrong number.

$n=3$ (numbr of digits)

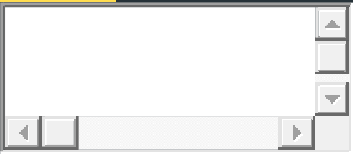
nth powers of the digits – (111), (555), (333)

The number should equal the nth power of their digits, i.e

$153 = (111) + (555) + (333).$

Other Armstrong Numbers: 153, 370, 371, 407, 1634, 8208, 9474

Python



```
# python program to check armstrong number

#Taking the input from user to check armstrong number
num=int(input("Enter the number to check armstrong number: "))

#Assigning the num value to arms
arms = num

#Finding the length of the number
length = len(str(num))

sum1 = 0

#Iterating the values to check armstrong number
while num != 0:

    rem = num % 10

    sum1 = sum1+(rem**length)

    num = num//10

#Printing the result whether the given number is armstrong number or not
if arms == sum1:

    print("The given number", arms, "is armstrong number")
else:

    print("The given number", arms, "is not an armstrong number")
```

Output1 of the Python armstrong number program

Enter the number to check armstrong number: 153

The given number 153 is armstrong number

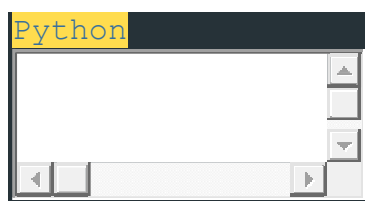
Output2 of the Python armstrong number program

Enter the number to check armstrong number: 123

The given number 123 is not an armstrong number

12) Write a Python program to check leap year.

Do you know what is the leap year, it occurs once every 4 years. It contains additional days which makes a year 366 days.



```
1 #Python program to check whether the given year is leap year or not
2
3
4 # Function implementation to check leap year
5 def LeapYear(Year):
6     #Condition to check if the given year is leap year or not
7     if((Year % 400 == 0) or
8        (Year % 100 != 0) and
9        (Year % 4 == 0)):
10        print("The given Year is a leap year");
11    # Else it is not a leap year
12    else:
13        print ("The given Year is not a leap year")
```

```

# Taking an input year from user
Year = int(input("Enter the year to check whether a leap year or not: "))
# Printing the leap year result
LeapYear(Year)

```

Output1 of the Python leap year program

Enter the year to check whether a leap year or not: 2020
The given Year is a leap year

Output2 of the Python leap year program

Enter the year to check whether a leap year or not: 2021
The given year is not a leap year

13) Write a Python program to check prime numbers.

A prime number is a positive integer $p > 1$ that has no positive integer divisors other than 1 and p itself.

Example Prime Numbers: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37,

```

Python
#Python program to check prime number

#Get the input to check whether a number is Prime or not

prime_num = int(input("Enter a number to check whether it is prime or not: "))

```

```

if prime_num > 1:
    for i in range(2, int(prime_num/2)+1):
        if (prime_num % i) == 0:
            print(prime_num, "is not a prime number")
            break
    else:
        print(prime_num, "is a prime number")
else:
    print(prime_num, "is not a prime number")

```

Output1 of Python program to check prime number

Enter a number to check whether it is prime or not: 7
7 is a prime number

Output2 of Python program to check prime number

Enter a number to check whether it is prime or not: 10
10 is not a prime number

14) Write a Python program to display a multiplication table using for loop.

Here is the Python program to display the multiplication table of any number using for loop.

```

Python
#Python program to display multiplication table

```

```
#Get the number from the user for multiplication table
```

```
tab_number = int(input("Enter the number of your choice to print the multiplication table: "))
```

```
#For loop to iterate the multiplication 10 times and print the table
```

```
print("The Multiplication Table of: ", tab_number)
```

```
for count in range(1, 11):
```

```
    print(tab_number, 'x', count, '=', tab_number * count)
```

The output of Python multiplication table

Enter the number of your choice to print the multiplication table: 10

The Multiplication Table of: 10

10 x 1 = 10

10 x 2 = 20

10 x 3 = 30

10 x 4 = 40

10 x 5 = 50

10 x 6 = 60

10 x 7 = 70

10 x 8 = 80

10 x 9 = 90

10 x 10 = 100

15) Write a Python program to swap two variables.

Python program to swap two variables.

Python

```
# Take inputs from the user to swap the two variables
```

```
num1 = input('Enter the first variable: ')
```

```

num2 = input('Enter the second variable: ')
4
#Printing the numbers before swap
5
print('The value of num1 before swapping: {}'.format(num1))
6
print('The value of num2 before swapping: {}'.format(num2))
7
8
#Use temporary variable and swap the values
9
temp = num1
10
num1 = num2
11
num2 = temp
12
13
#Printing the numbers after swap
14
print('The value of num1 after swapping: {}'.format(num1))
15
print('The value of num2 after swapping: {}'.format(num2))
16
17

```

The output of the Python program to swap two numbers:

Enter the first variable: 5

Enter the second variable: 10

The value of num1 before swapping: 5

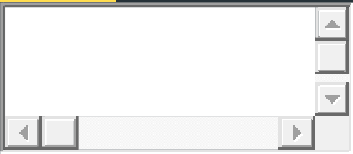
The value of num2 before swapping: 10

The value of num1 after swapping: 10

The value of num2 after swapping: 5

16) Write a Python program to check if a given number is a Fibonacci number or not.

Here is the Python program to check the Fibonacci number:



```
#Python program to check fibonacci number

# First two fibonacci terms

fib_nums = [0, 1]

number = int(input('Enter the number you want to check for fibonacci number: '))

# Iterate fibonacci terms until the user number is reached

while fib_nums[-1] <= number:

    fib_nums.append(fib_nums[-1] + fib_nums[-2])

#Printing the result whether the given number is fibonacci or not

if number in fib_nums:

    print(f'Yes. {number} is a fibonacci number.')

else:

    print(f'No. {number} is NOT a fibonacci number.')
```

Output1 of Python program to check Fibonacci number

Enter the number you want to check for fibonacci number: 8
Yes. 8 is a fibonacci number.

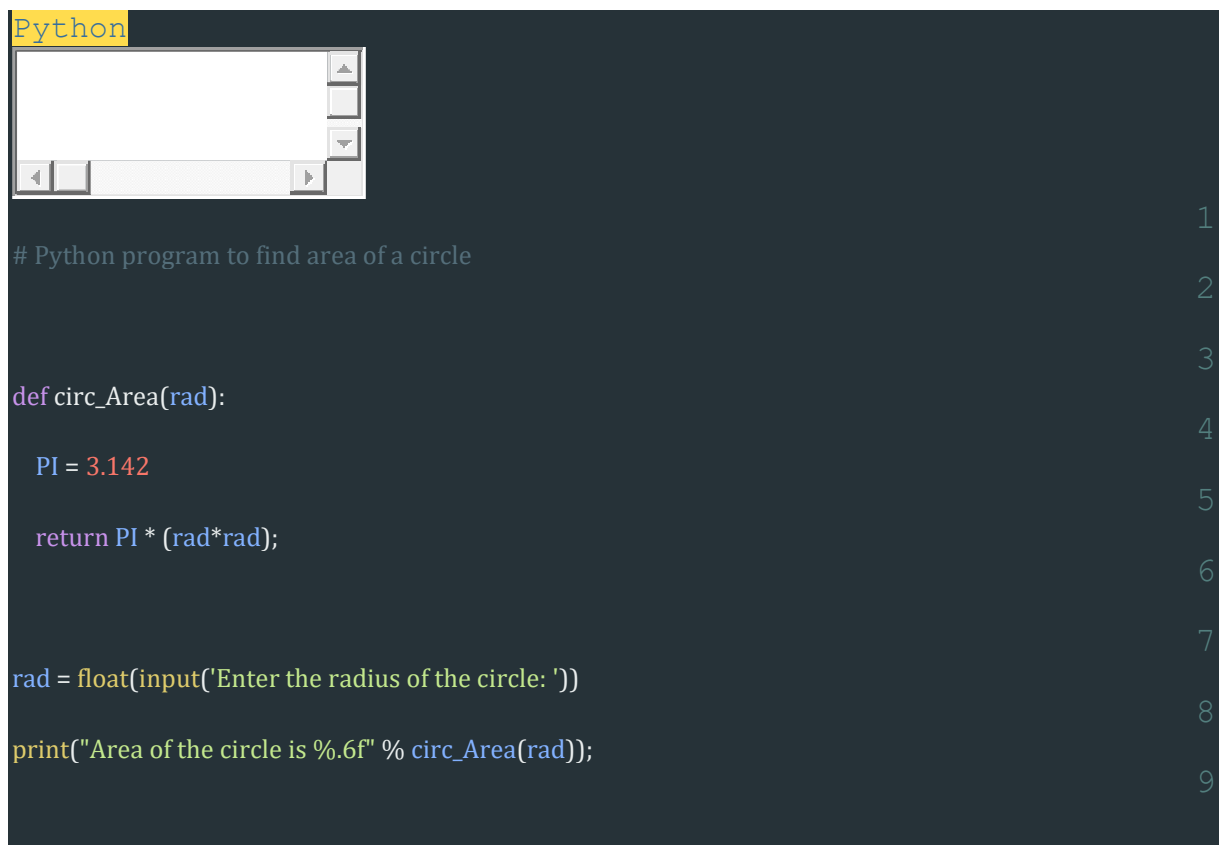
Output2 of Python program to check Fibonacci number

Enter the number you want to check for fibonacci number: 10
No. 10 is NOT a fibonacci number.

17) Write a Python program to find the area of a circle.

The formula to find the area of the circle is: $\text{Area} = \pi * r^2$, where r is the radius of the circle.

Here is the Python program to find the area of the circle.

A screenshot of a Python IDE window. The title bar says "Python". The code editor has a dark background with light-colored text. The code is as follows:

```
# Python program to find area of a circle

def circ_Area(rad):

    PI = 3.142

    return PI * (rad*rad);

rad = float(input('Enter the radius of the circle: '))

print("Area of the circle is %.6f" % circ_Area(rad));
```

Line numbers 1 through 9 are visible on the right side of the editor.

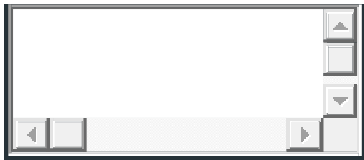
The output of the Python program is to find the area of a circle.

Enter the radius of the circle: 4
The area of the circle is 50.272000

18) Write a Python program to display the calendar.

Python program to display the calendar.

A screenshot of the top part of a Python IDE window, showing the title bar with the word "Python" in a yellow box.



```
#Python program to display calendar  
  
import calendar  
  
# Get the month and year from the users  
  
year = int(input("Enter the year: "))  
month = int(input("Enter the month: "))  
  
# Displaying the calendar for the given year and month  
  
print(calendar.month(year,month))
```

The output of the Python program to display the calendar.

```
Enter the year: 2022  
Enter the month: 10  
October 2022  
Mo Tu We Th Fr Sa Su  
1 2  
3 4 5 6 7 8 9  
10 11 12 13 14 15 16  
17 18 19 20 21 22 23  
24 25 26 27 28 29 30  
31
```

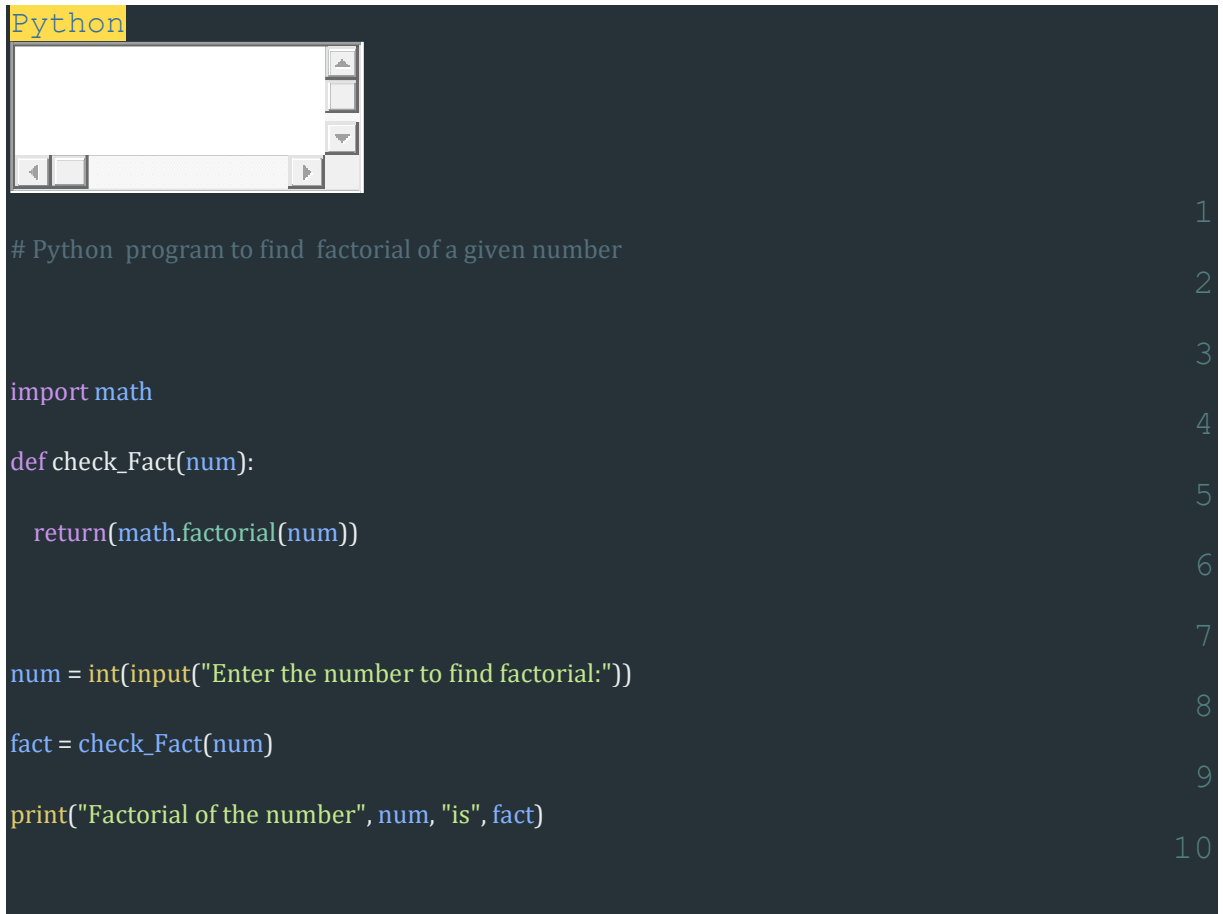
19) Write a Python program to find factorial of the given number.

The factorial formula:

$$n! = n * (n-1) * (n-2) * \dots * 1$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

A screenshot of a Python IDE window. The title bar says "Python". The editor area contains a Python program to calculate the factorial of a number. The program includes a comment, an import statement, a function definition, and several lines of code for input, function call, and output. Line numbers 1 through 10 are visible on the right side of the editor.

```
1 # Python program to find factorial of a given number
2
3
4 import math
5
6 def check_Fact(num):
7     return(math.factorial(num))
8
9 num = int(input("Enter the number to find factorial:"))
10 fact = check_Fact(num)
11 print("Factorial of the number", num, "is", fact)
```

The output of the Python factorial program

Enter the number to find factorial:5
Factorial of the number 5 is 120

20) Write a Python program to print all prime numbers in an interval.

Here is the Python program to display all the prime numbers within a range.

A screenshot of a Python IDE window. The title bar says "Python". The editor area is mostly empty, with only line number 1 visible on the right side.

```
1
```

```
# Python program to print all prime number in an interval
```

```
def disp_Prime(num1, num2):
```

```
    prime_List = []
```

```
    for i in range(num1, num2):
```

```
        if i == 0 or i == 1:
```

```
            continue
```

```
        else:
```

```
            for j in range(2, int(i/2)+1):
```

```
                if i % j == 0:
```

```
                    break
```

```
            else:
```

```
                prime_List.append(i)
```

```
    return prime_List
```

```
# Driver program
```

```
starting_Num = int(input('Enter the starting range: '))
```

```
ending_Num = int(input('Enter the ending range: '))
```

```
lst = disp_Prime(starting_Num, ending_Num)
```

```
if len(lst) == 0:
```

```
    print("There are no prime numbers in this range")
```

```
else:
```

```
    print("The prime numbers in the given range are: ", lst)
```

The output of Python prime numbers display program

Enter the starting range: 5

Enter the ending range: 25

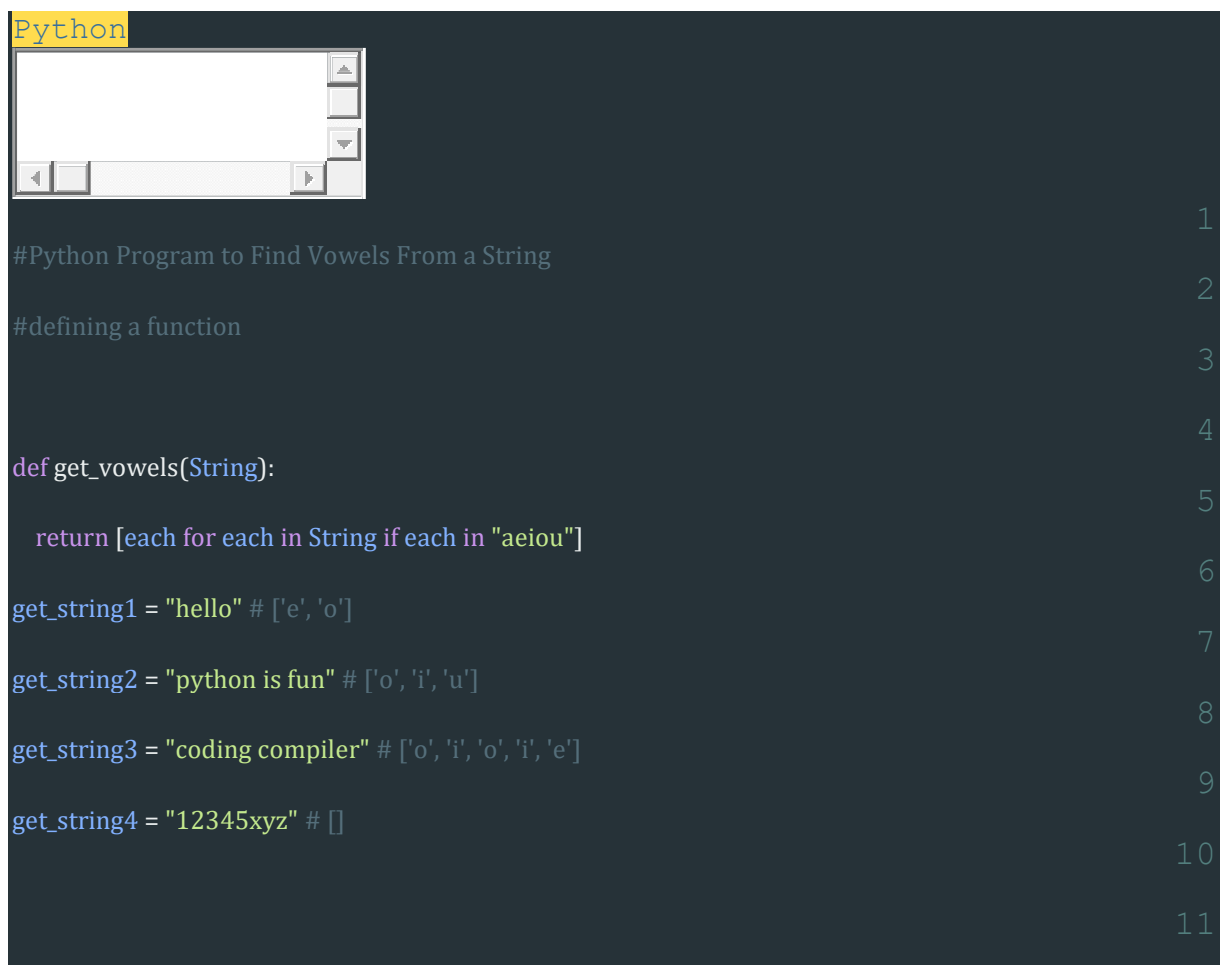
The prime numbers in this range are: [5, 7, 11, 13, 17, 19, 23]

Python Strings Interview Questions & Answers

21) Write a Python Program to Find Vowels From a String.

Python Program to Find Vowels From a String

This Python coding example returns the vowels "a e i o u" present in a string. This Python program is useful when finding vowels, Let's try this and see how it works.

A screenshot of a Python IDE with a dark background. The title bar of the editor window says "Python". The code is as follows:

```
#Python Program to Find Vowels From a String

#defining a function

def get_vowels(String):

    return [each for each in String if each in "aeiou"]

get_string1 = "hello" # ['e', 'o']

get_string2 = "python is fun" # ['o', 'i', 'u']

get_string3 = "coding compiler" # ['o', 'i', 'o', 'i', 'e']

get_string4 = "12345xyz" # []
```

Line numbers 1 through 11 are visible on the right side of the code editor.

```

#Let's print vowels from the given strigns
#Vowels from first string
print("The Vowels Are: ",get_vowels(get_string1))

#Vowels from second string
print("The Vowels Are: ",get_vowels(get_string2))

#Vowels from third string
print("The Vowels Are: ",get_vowels(get_string3))

#Vowels from fourth string
print("The Vowels Are: ",get_vowels(get_string4))

```

The output of the above Python code:

```

The Vowels Are: ['e', 'o']
The Vowels Are: ['o', 'i', 'u']
The Vowels Are: ['o', 'i', 'o', 'i', 'e']
The Vowels Are: []

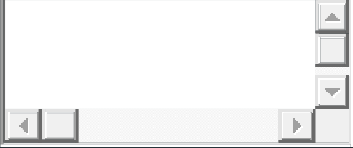
```

22) Write a Python Program to Convert Comma Separated List to a String.

Python Program to Convert Comma Separated List to a String

In this Python coding example we use the `join()` method to convert comma separated list to a string. Let's try this Python program and see how it works.

Python



```
# Python program to converting a comma separated list to a string

#comma seperated list

favorite_prog = ["Python", "SQL", "GO"]

#The join() method takes all items from the favorite_prog list and joins them into one string.

print("What is your favorite programming language:", ", ".join(favorite_prog))
```

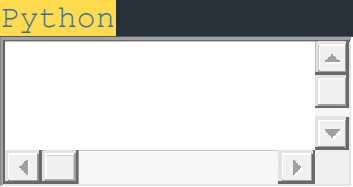
The output of the above Python code:

What is your favorite programming language: Python, SQL, GO

23) Write a Python Program to Capitalize the First Letter of a String.

Python program to convert every first letter of the characters in a string to an uppercase.

This Python coding example capitalizes every first letter of the characters in a string. This Python program is useful when converting the first letter of a string to a capital letter, Let's try this and see how it works.



```
Python

#Python Program to Capialize the First Letter of a String

#defining a capitalize function
```

```

def capitalize(String):
    return String.title()

get_capital1 = "hello" # [Hello]

get_capital2 = "python programming" # [Python Programming]

get_capital3 = "python is easy to learn" # [Python Is Easy To Learn]

#Let's print capitalized string from the given strings

#Capitalized string from first string

print("The Capitalized String Is: ",capitalize(get_capital1))

#Capitalized string from second string

print("The Capitalized String Is: ",capitalize(get_capital2))

#Capitalized string from third string

print("The Capitalized String Is: ",capitalize(get_capital3))

```

The output of the above Python code:

The Capitalized String Is: Hello
 The Capitalized String Is: Python Programming
 The Capitalized String Is: Python Is Easy To Learn

Python Implicit/Explicit Type Conversion Interview Questions

24) Write a Python Program to Implicit Type Conversion.

Python Program to Demonstrate Implicit Type Conversion.

This Python coding example explains implicit type conversion in Python. This Python program is useful when working with different data types and conversions. Let's try this and see how it works.

A screenshot of a Python IDE window. The title bar says "Python". The code editor contains the following Python code:

```
#Implicit type casting example

get_num1 = 199 #int

get_num2 = 1.25 #float

get_num3 = get_num1 + get_num2 #int+float=?

print("Datatype of get_num1:",type(get_num1))

print("Datatype of get_num2:",type(get_num2))

print("The value of get_num3 is:",get_num3)

print("Datatype of get_num3:",type(get_num3))

#Implicit conversion - Python always converts smaller data types to larger data types to avoid the loss of data.
```

Line numbers 1 through 12 are visible on the right side of the code editor.

The output of the above Python code:

Datatype of get_num1:

Datatype of get_num2:

The value of get_num3 is: 200.25
Datatype of get_num3: <class 'float'>

25) What's the Output of this Python program?

Let's find out what's the output of the below Python program.



```
Python
#Let's find out what's the output of the below Python program.

#Implicit type casting example

num1 = 123

num2 = "456"

print("Data type of num_int:",type(num1))

print("Data type of num_str:",type(num2))

print("num1 data type is:",type(num1 + num2))
```

The output of the above Python code:

Data type of num_int:
Data type of num_str:
Traceback (most recent call last):
File "", line 11, in
TypeError: unsupported operand type(s) for +: 'int' and 'str'

Python is not able to convert string type to integer type implicitly. So we can see TypeError in output.

26) Write a Python Program to Explicit Type Conversion.

Python Program to Demonstrate Explicit Type Conversion.

Python coding example explains explicit type conversion in Python. This Python program is useful when working with different data types and conversions. Let's try this Python code and see how it works.

A screenshot of a Python IDE window. The title bar says "Python". The code is as follows:

```
#Python Program to Demonstrate Explicit Type Conversion

#Explicit type casting example

num1 = 123

num2 = "456"

print("Datatype of num1:",type(num1))

print("Datatype of num2 before Type Casting:",type(num2))

#Conversion of num2 from string(higher) to integer(lower) type using int() function to perform the
addition between num1 and num2.

num2 = int(num2)

print("Datatype of num2 after Type Casting:",type(num2))

num3 = num1 + num2
```

Line numbers 1 through 16 are visible on the right side of the code editor.

```
print("Sum of num1 and num2:",num3)
print("Data type of the num3:",type(num3))
```

17
18
19

The output of the above Python code:

Datatype of num1: <class 'int'>

Datatype of num2 before Type Casting: <class 'str'>

Datatype of num2 after Type Casting: <class 'int'>

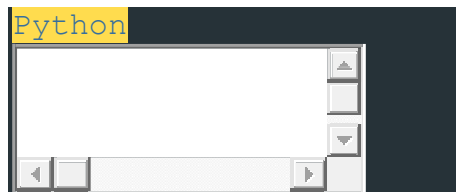
Sum of num1 and num2: 579

Data type of the num3: <class 'int'>

Python Namespace Coding Interview Questions

27) What's the Output of this Python Namespace program?

Python Namespace Program.



```
#Python Namespace Program Example
```

```
def outer_function():
```

```
    num = 20
```

```
    def inner_function():
```

1
2
3
4
5
6

```

num = 30
print('num =', num)

inner_function()

print('num =', num)

num = 10
outer_function()

print('num =', num)

```

The output of the above Python code:

Three different variables "num" are defined in separate namespaces and accessed accordingly.

```

num = 30
num = 20
num = 10

```

28) What's the Output of this Python program?

Python Namespace Program.

```

Python
#Python Namespace Program Example

```

```

def outer_function():
    global num
    num = 20

    def inner_function():
        global num
        num = 30
        print('num =', num)

    inner_function()
    print('num =', num)

num = 10
outer_function()
print('num =', num)

```

The output of the above Python code:

All references and assignments are to the global 'num' due to the use of the keyword 'global'.

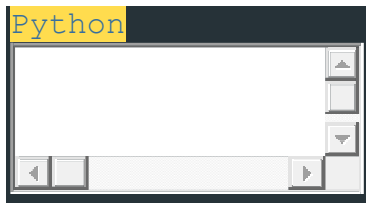
```

num = 30
num = 30
num = 30

```

29) What does this do, and why should one include the if statement?

Explain what the below Python program does.



```
Python
if __name__ == "__main__":
    print("Hello, World!")
```

1
2
3

Answer#

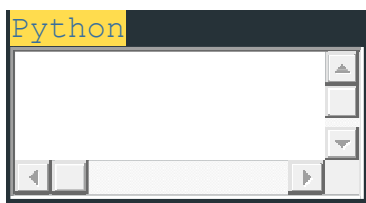
It's boilerplate code that protects users from accidentally invoking the script when they didn't intend to. Here are some common problems when the guard is omitted from a script:

If you import the guardless script in another script (e.g. `import my_script_without_a_name_eq_main_guard`), then the latter script will trigger the former to run at import time and use the second script's command line arguments. This is almost always a mistake.

If you have a custom class in the guardless script and save it to a pickle file, then unpickling it in another script will trigger an import of the guardless script, with the same problems outlined in the previous bullet.

Example Python Code:

Create the following two files:



```
Python
# a.py
import b
```

1
2
3
4

```

# b.py
5
6
print("__name__ equals " + __name__)
7
8
if __name__ == '__main__':
9
    print("if-statement was executed")
10
Now run each file individually.
11
12
Running python a.py:
13
14
$ python a.py
15
__name__ equals b
16
When a.py is executed, it imports the module b. This causes all the code inside b to run. Python sets
globals()['__name__'] in the b module to the module's name, b.
17
18
19
#Running python b.py:
20
21
$ python b.py
22
__name__ equals __main__
23

```

if-statement was executed, when only the file b.py is executed, Python sets `globals()['name']` in this file to **"main"**. Therefore, the if statement evaluates to True this time.

30) What does `__all__` mean in Python?

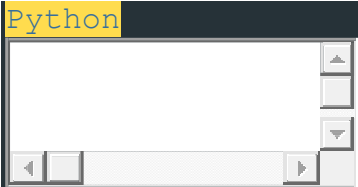
Answer#

It's a list of public objects of that module, as interpreted by `import *`. It overrides the default of hiding everything that begins with an underscore.

Example Python Code:

Linked to, but not explicitly mentioned here, is exactly when `__all__` is used. It is a list of strings defining what symbols in a module will be exported when `from <module> import *` is used on the module.

For example, the following code in a `foo.py` explicitly exports the symbols `bar` and `baz`:

A small window titled "Python" with a white background and a scrollbar, showing a portion of the code above the main editor.

```
__all__ = ['bar', 'baz']

waz = 5
bar = 10
def baz(): return 'baz'

# These symbols can then be imported like so:

from foo import *

print(bar)
print(baz)

# The following will trigger an exception, as "waz" is not exported by the module
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

```
print(waz)
```

17

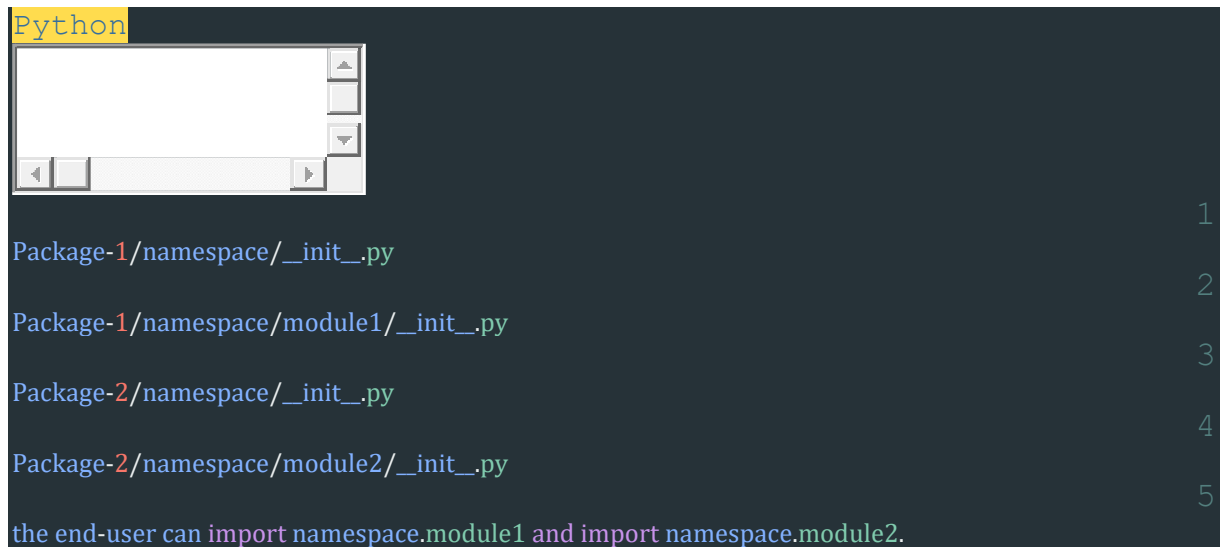
If the `__all__` above is commented out, this code will then execute to completion, as the default behavior of `import *` is to import all symbols that do not begin with an underscore, from the given namespace.

Reference: <https://docs.python.org/tutorial/modules.html#importing-from-a-package>

NOTE: `__all__` affects the `from <module> import *` behavior only. Members that are not mentioned in `__all__` are still accessible from outside the module and can be imported with `from <module> import <member>`.

31) How to create a namespace package in Python?

In Python, a namespace package allows you to spread Python code among several projects. This is useful when you want to release related libraries as separate downloads. For example, with the directories `Package-1` and `Package-2` in `PYTHONPATH`,



```
Package-1/namespace/__init__.py
Package-1/namespace/module1/__init__.py
Package-2/namespace/__init__.py
Package-2/namespace/module2/__init__.py
the end-user can import namespace.module1 and import namespace.module2.
```

What's the best way to define a namespace package so more than one Python product can define modules in that namespace?

Answer#

On Python 3.3 you don't have to do anything, just don't put any `__init__.py` in your namespace package directories and it will just work. On pre-3.3, choose the `pkgutil.extend_path()` solution over the `pkg_resources.declare_namespace()` one, because it's future-proof and already compatible with implicit namespace packages.

Python 3.3 introduces implicit namespace packages.

This means there are now three types of objects that can be created by an `import foo`:

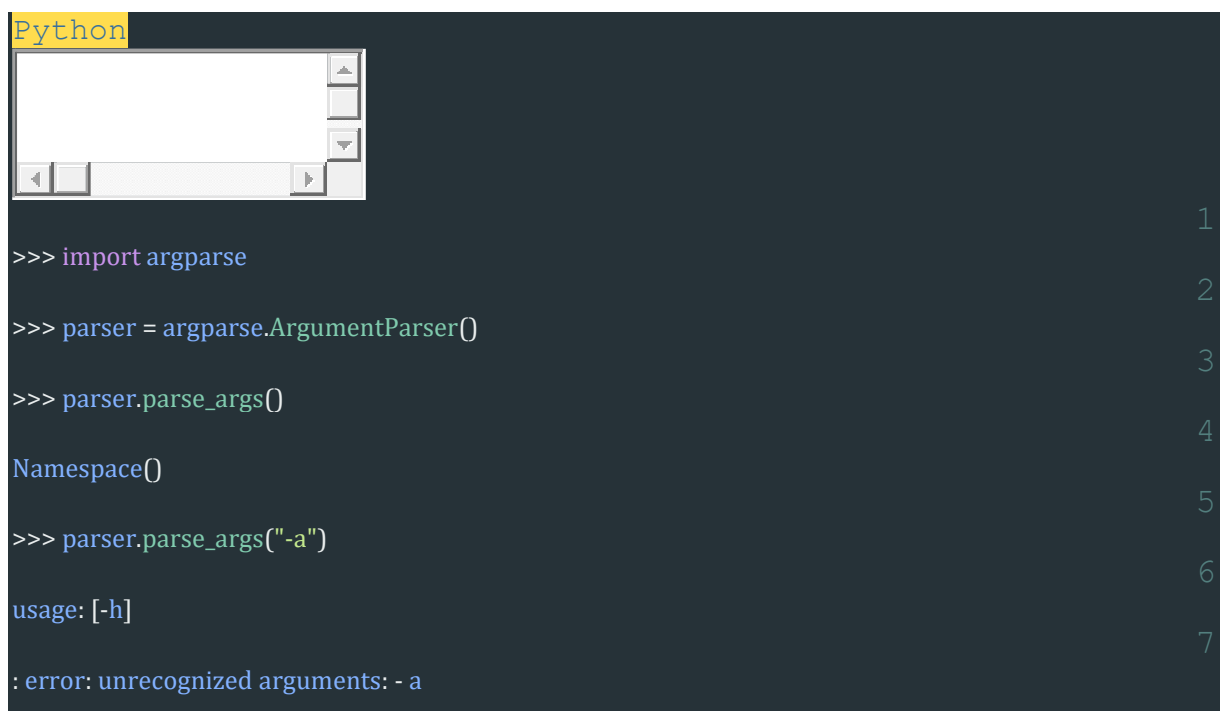
A module represented by a `foo.py` file

A regular package, represented by a directory `foo` containing an `__init__.py` file

A namespace package, represented by one or more directories `foo` without any `__init__.py` files

32) How to create a Python namespace (`argparse.parse_args` value)?

To interactively test my python script, I would like to create a Namespace object, similar to what would be returned by `argparse.parse_args()`. The obvious way,



```
Python
>>> import argparse
>>> parser = argparse.ArgumentParser()
>>> parser.parse_args()
Namespace()
>>> parser.parse_args("-a")
usage: [-h]
: error: unrecognized arguments: - a
```

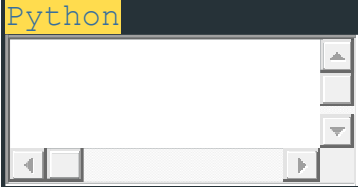
Process Python exited abnormally with code 2 may result in Python repl exiting (as above) on a silly error.

So, what is the easiest way to create a Python namespace with a given set of attributes?

E.g., I can create a dict on the fly (`dict([("a",1),("b","c")])`) but I cannot use it as a Namespace:

`AttributeError: 'dict' object has no attribute 'a'`

Answer#



```
#You can create a simple class:

class Namespace:

    def __init__(self, **kwargs):

        self.__dict__.update(kwargs)

#and it'll work the exact same way as the argparse Namespace class when it comes to attributes:

>>> args = Namespace(a=1, b='c')

>>> args.a
1

>>> args.b
'c'

#Alternatively, just import the class; it is available from the argparse module:
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

```
from argparse import Namespace
```

```
args = Namespace(a=1, b='c')
```

As of Python 3.3, there is also `types.SimpleNamespace`, which essentially does the same thing:

```
>>> from types import SimpleNamespace
```

```
>>> args = SimpleNamespace(a=1, b='c')
```

```
>>> args.a
```

```
1
```

```
>>> args.b
```

```
'c'
```

The two types are distinct; `SimpleNamespace` is primarily used for the `sys` implementation attribute and the return value of `time.get_clock_info()`.

Further comparisons:

- Both classes support equality testing; for two instances of the same class, `instance_a == instance_b` is true if they have the same attributes with the same values.
- Both classes have a helpful `__repr__` to show what attributes they have.
- `Namespace()` objects support containment testing; `'attrname' in instance` is true if the namespace instance has an attribute named `attrname`. `SimpleNamespace` does not.
- `Namespace()` objects have an undocumented `._get_kwargs()` method that returns a sorted list of (name, value) attributes for that instance. You can get the same for either class using `sorted(vars(instance).items())`.
- While `SimpleNamespace()` is implemented in C and `Namespace()` is implemented in Python, attribute access is no faster because both use the same `__dict__` storage for the attributes. Equality testing and

producing the representation are a little faster for SimpleNamespace() instances.

33) What

does `__import__('pkg_resources').declare_namespace(__name__)` do?

In some **init.py** files of modules I saw such single line:

```
import('pkg_resources').declare_namespace(name)
```

What does it do and why people use it? Suppose it's related to dynamic importing and creating namespace at runtime.

Answer#

Here are two things:

1) **__import__** is a Python function that will import a package using a string as the name of the package. It returns a new object that represents the imported package. So `foo = __import__('bar')` will import a package named bar and store a reference to its objects in a local object variable foo.

2 From setup utils pkg_resources' documentation, `declare_namespace()` "Declare[s] that the dotted package name is a "namespace package" whose contained packages and modules may be spread across multiple distributions."

So `__import__('pkg_resources').declare_namespace(__name__)` will import the 'pkg_resources' package into a temporary and call the `declare_namespace` function stored in that temporary (the **__import__** function is likely used rather than the import statement so that there is no extra symbol left over named pkg_resources). If this code were in `my_namespace/__init__.py`, then **__name__** is `my_namespace` and this module will be included in the `my_namespace` namespace package.

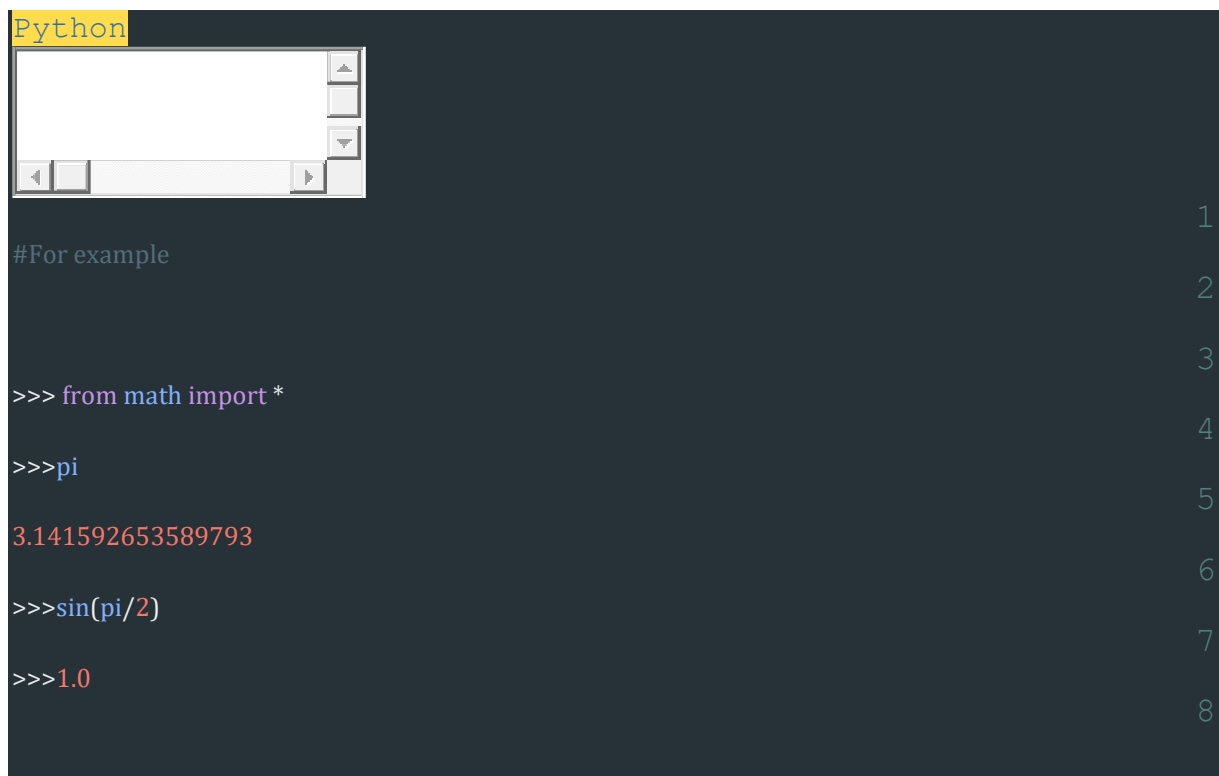
34) What exactly does "import *" import?

In Python, what exactly does `import *` import? Does it import `__init__.py` found in the containing folder?

For example, is it necessary to declare `from project.model import __init__`, or is `from project.model import *` sufficient?

Answer#

The “advantage” of `from xyz import *` as opposed to other forms of `import` is that it imports everything (well, almost... [see (a) below] everything) from the designated module under the current module. This allows using the various objects (variables, classes, methods...) from the imported module without prefixing them with the module’s name.

A screenshot of a Python REPL window. The title bar says "Python". The window contains a text area with the following text:

```
#For example

>>> from math import *

>>> pi

3.141592653589793

>>> sin(pi/2)

>>> 1.0
```

 To the right of the text area, there are line numbers 1 through 8. The background is dark grey, and the text is light grey. The window has standard OS controls (minimize, maximize, close) in the top right corner.

35) Why is “import *” bad in Python?

It is recommended to not to use `import *` in Python.

Can you please share the reason for that?

Answer#

Because it puts a lot of stuff into your namespace (might shadow some other object from the previous import and you won’t know about it).

Because you don't know exactly what is imported and can't easily find from which module a certain thing was imported (readability).

Because you can't use cool tools like pyflakes to statically detect errors in your code.

36) What are Python namespaces and explain how to use namespaces & how this feature makes programming better?

Answer#

Namespace is a way to implement scope.

In Java (or C) the compiler determines where a variable is visible through static scope analysis.

In C, scope is either the body of a function or it's global or it's external. The compiler reasons this out for you and resolves each variable name based on scope rules. External names are resolved by the linker after all the modules are compiled.

In Java, scope is the body of a method function, or all the methods of a class. Some class names have a module-level scope, also. Again, the compiler figures this out at compile time and resolves each name based on the scope rules.

In Python, each package, module, class, function and method function owns a "namespace" in which variable names are resolved. Plus there's a global namespace that's used if the name isn't in the local namespace.

Each variable name is checked in the local namespace (the body of the function, the module, etc.), and then checked in the global namespace.

Variables are generally created only in a local namespace. The global and nonlocal statements can create variables in other than the local namespace.

When a function, method function, module or package is evaluated (that is, starts execution) a namespace is created. Think of it as an "evaluation context".

When a function or method function, etc., finishes execution, the namespace is dropped. The variables are dropped. The objects may be dropped, also.

37) Is it a good idea to use the class as a namespace in Python?

Answer#

Yes, indeed. You can use Python classes strictly for namespacing as that is one of the special things they can do and do differently than modules. It's a lot easier to define a class as a namespace inline in a file than to generate more files.

You should not do it without commenting on your code saying what it's for. Python classes come in a lot of different forms and purposes and this makes it difficult to understand code you have not seen before.

A Python class used as a namespace is no less a Python class than one that meets the perception of what a class is in other languages. Python does not require a class to be instantiated to be useful. It does not require ivars and does not require methods. It is fairly flexible.

Classes can contain other classes too.

Lots of people have their ideas about what is or isn't Pythonic. But if they were all worried about something like consistency, they'd push to have things like `len()` `dir()`, and `help()` be a method of objects rather than a global function.

38) Is it possible to add an object to the global namespace, for example, by using `globals()` or `dir()`?



```
def insert_into_global_namespace(var_name, value):
```

```
    globals()[var_name] = value
```

```
insert_into_global_namespace('my_obj', 'an object')
```

```
print(f'my_obj = {my_obj}')
```

```
#But this only works in the current module.
```

Answer#

Python

```
#It is as simple as
```

```
globals()['var'] = "an object"
```

```
and/or
```

```
def insert_into_namespace(name, value, name_space=globals()):
```

```
    name_space[name] = value
```

```
insert_into_namespace("var", "an object")
```

```
# Remark that globals is a built-in keyword, that is, 'globals' in __builtins__.__dict__ evaluates to True.
```

39) What's the python `__all__` module level variable for?

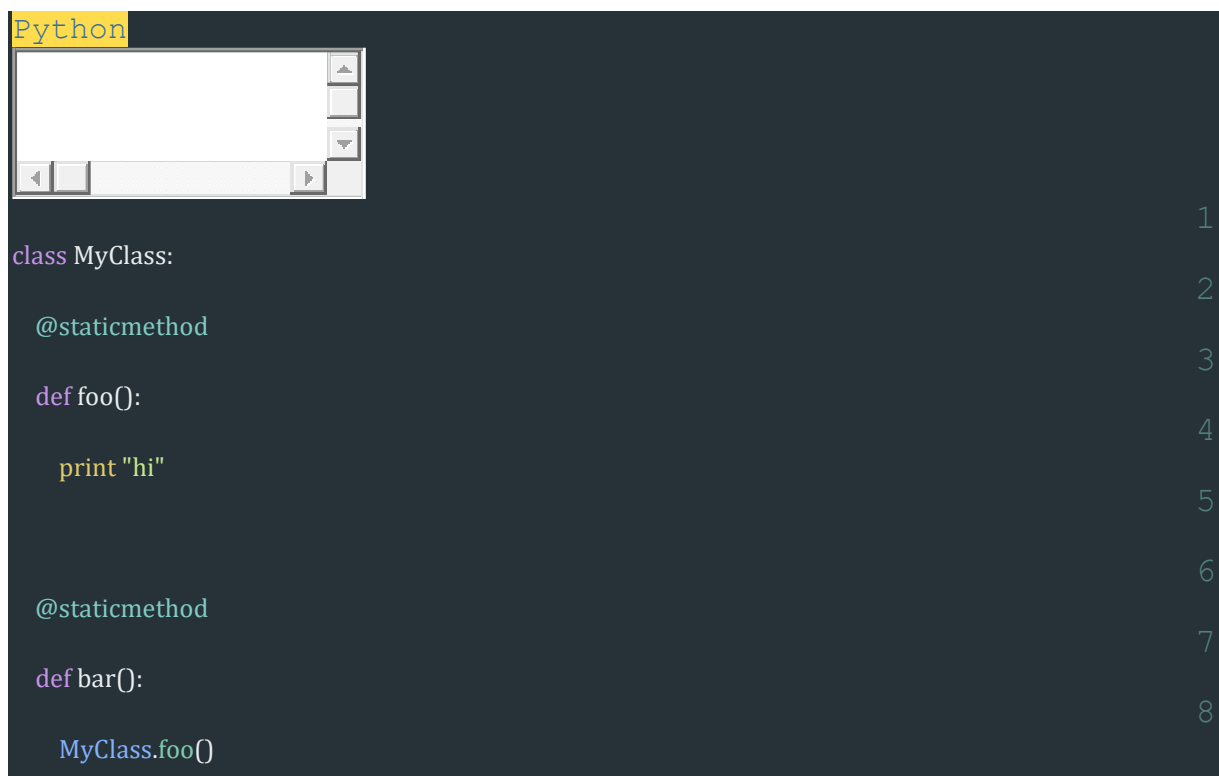
Answer#

It has two purposes:

Anybody who reads the source will know what the exposed public API is. It doesn't prevent them from poking around in private declarations but does provide a good warning not to.

When using `from mod import *`, only names listed in `__all__` will be imported. This is not as important, in my opinion, because importing everything is a really bad idea.

40) Is it possible to call static method from within class without qualifying the name in Python?

A screenshot of a Python IDE with a dark background. The title bar of the editor window says 'Python'. The code is as follows:

```
class MyClass:
    @staticmethod
    def foo():
        print "hi"

    @staticmethod
    def bar():
        MyClass.foo()
```

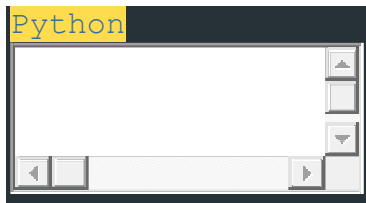
Line numbers 1 through 8 are visible on the right side of the code editor.

Is there a way to make this work without naming `MyClass` in the call? i.e. so I can just say `foo()` on the last line?

Answer#

There is no way to use `foo` and get what you want. There is no implicit class scope, so `foo` is either a local or a global, neither of which you want.

You might find classmethods more useful:

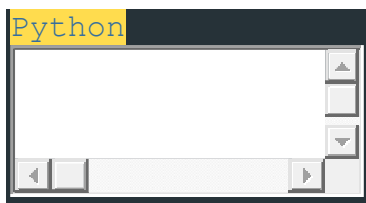
A screenshot of a Python IDE window with a dark background. The title bar says 'Python'. The code editor shows a class definition. Line numbers 1 through 10 are visible on the right side of the editor.

```
1 class MyClass:
2
3     @classmethod
4     def foo(cls):
5
6         print "hi"
7
8     @classmethod
9     def bar(cls):
10
11         cls.foo()
```

#This way, at least you don't have to repeat the name of the class.

Object-Oriented Python Coding Interview Questions

41) What is the output of the below
(`__class__.__name__`) Object Oriented Python
code?

A screenshot of a Python IDE window with a dark background. The title bar says 'Python'. The code editor shows a class definition. Line numbers 1 and 2 are visible on the right side of the editor.

```
1 class Programming:
2
3     def name(self, name):
```

```

    return name
3
4
5
p = Programming()
6
print(p.__class__.__name__)

```

#Output

Programming

42) What's the output of the below (type().__name__) Python code?

```

Python
class Programming:
1
2
    def name(self, name):
3
4
        return name
5
p = Programming()
6
print(type(p).__name__)

```

#Output

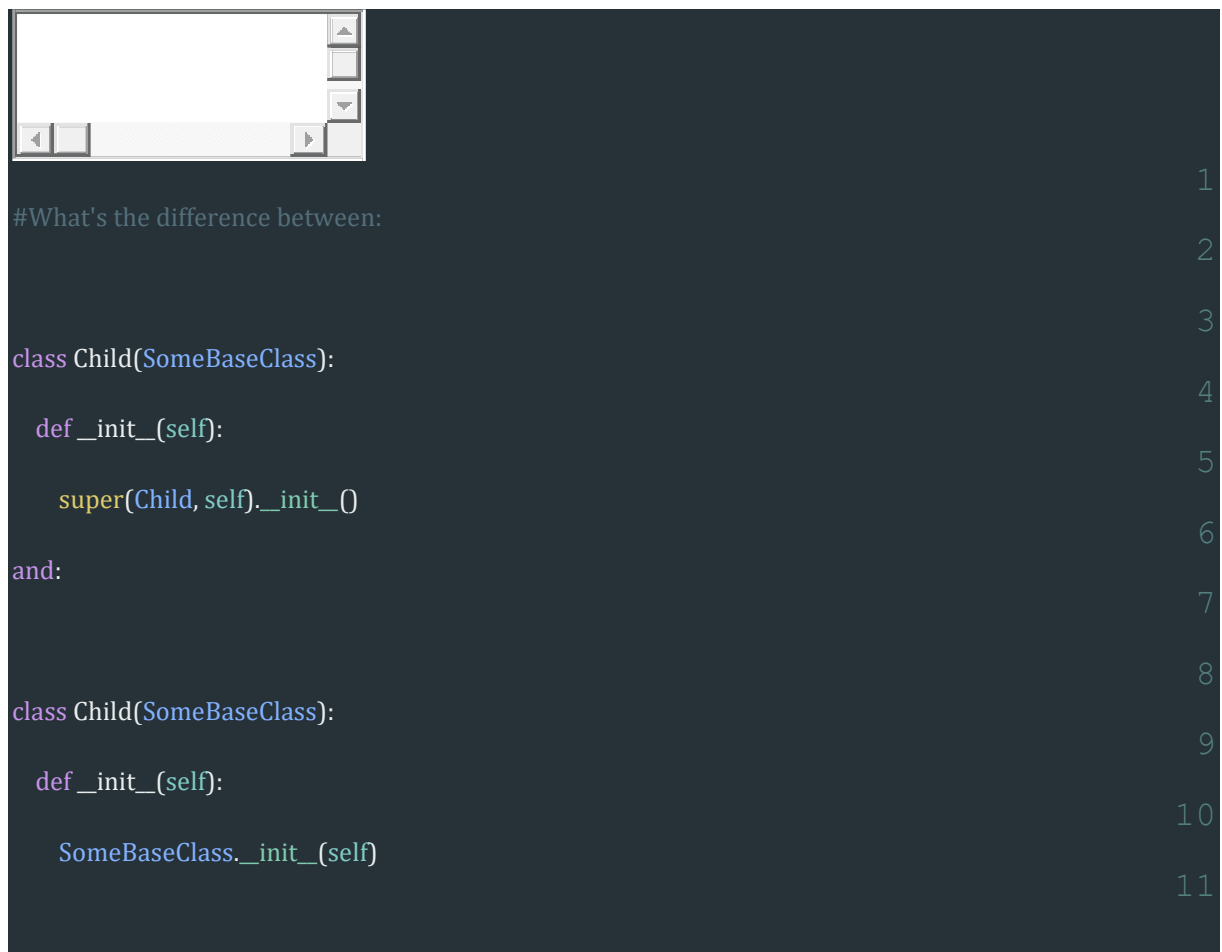
Programming

43) What does super() do in Python? The difference between super().__init__() and explicit superclass __init__()?

```

Python

```



```
1 #What's the difference between:
2
3
4 class Child(SomeBaseClass):
5     def __init__(self):
6         super(Child, self).__init__()
7
8 and:
9
10 class Child(SomeBaseClass):
11     def __init__(self):
12         SomeBaseClass.__init__(self)
```

I've seen `super` being used quite a lot in classes with only single inheritance. I can see why you'd use it in multiple inheritance, but I'm unclear as to what the advantages are of using it in this kind of situation.

#Output

`super()` lets you avoid referring to the base class explicitly, which can be nice. But the main advantage comes with multiple inheritance, where all sorts of fun stuff can happen. See the standard docs on `super` if you haven't already.

Note that the syntax changed in Python 3.0: you can just say `super().__init__()` instead of `super(ChildB, self).__init__()` which IMO is quite a bit nicer.

What's the difference?

`SomeBaseClass.__init__(self)`
means to call `SomeBaseClass`'s `__init__`. while

`super().__init__()`

means to call a bound `__init__` from the parent class that follows SomeBaseClass's child class (the one that defines this method) in the instance's Method Resolution Order (MRO).

If the instance is a subclass of this child class, there may be a different parent that comes next in the MRO.

Explained simply

When you write a class, you want other classes to be able to use it. `super()` makes it easier for other classes to use the class you're writing.

Good architecture allows you to postpone decision-making as long as possible.

`super()` can enable that sort of architecture.

When another class subclasses the class you wrote, it could also be inherited from other classes. And those classes could have an `__init__` that comes after this `__init__` based on the ordering of the classes for method resolution.

Without `super` you would likely hard-code the parent of the class you're writing (like the example does). This would mean that you would not call the next `__init__` in the MRO, and you would thus not get to reuse the code in it.

If you're writing your own code for personal use, you may not care about this distinction. But if you want others to use your code, using `super` is one thing that allows greater flexibility for users of the code.

Python 2 versus 3

This works in Python 2 and 3:

`super(Child, self).__init__()`

This only works in Python 3:

`super().__init__()`

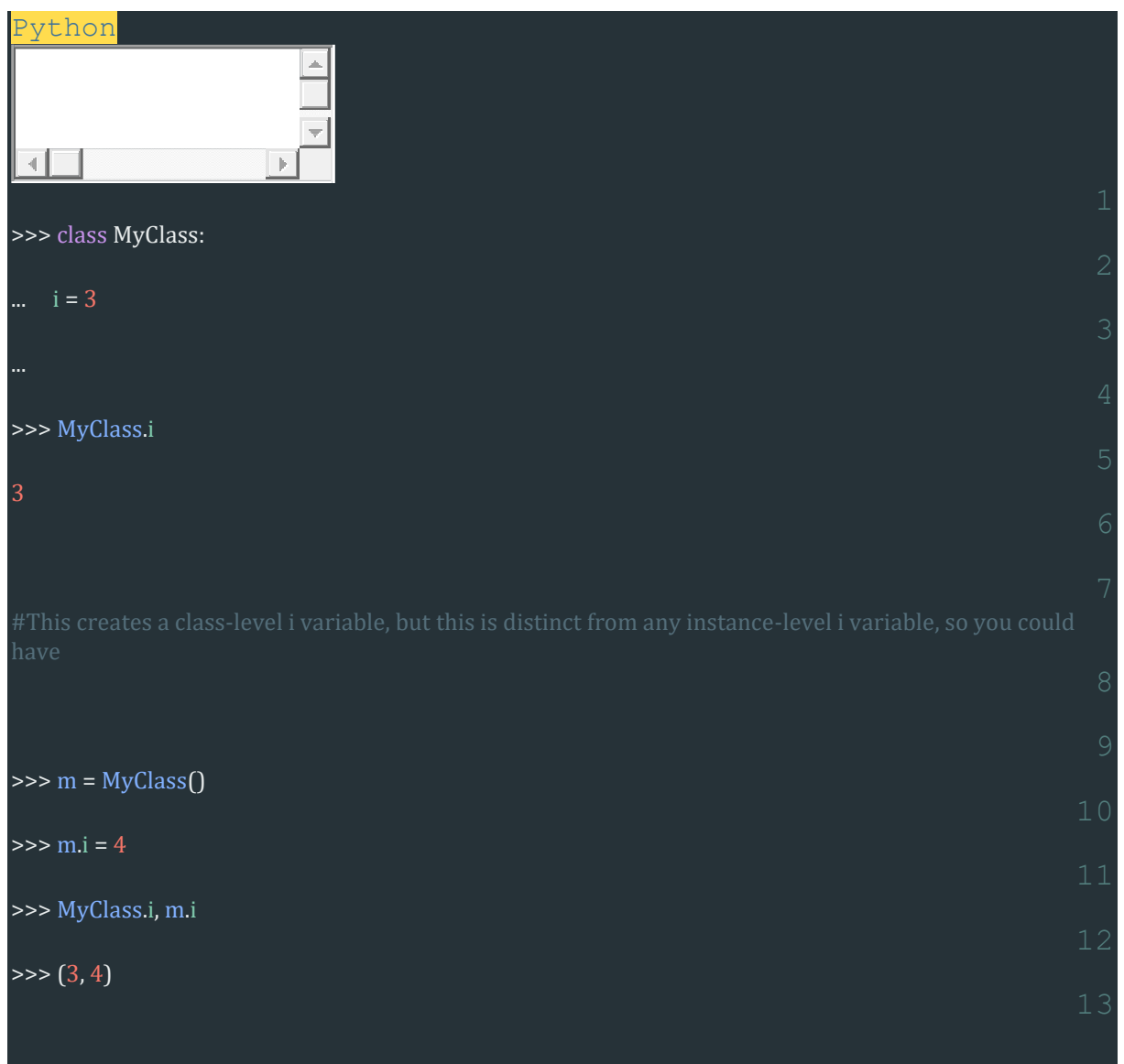
It works with no arguments by moving up in the stack frame and getting the first argument to the method (usually `self` for an instance method or `cls` for a

class method – but could be other names) and finding the class (e.g. Child) in the free variables (it is looked up with the name **class** as a free closure variable in the method).

44) How to create static class variables or methods in Python?

Answer#

Variables declared inside the class definition, but not inside a method are class or static variables:

A screenshot of a Python IDE window. The title bar says 'Python'. The editor has a dark background with syntax-highlighted Python code. Line numbers 1 through 13 are visible on the right side of the editor. The code defines a class 'MyClass' with a class-level variable 'i' set to 3. It then creates an instance 'm' of 'MyClass' and sets 'm.i' to 4. Finally, it prints the values of 'MyClass.i' and 'm.i', which are 3 and 4 respectively.

```
1 >>> class MyClass:
2 ...     i = 3
3 ...
4
5 >>> MyClass.i
6
7 3
8
9 #This creates a class-level i variable, but this is distinct from any instance-level i variable, so you could
10 have
11
12 >>> m = MyClass()
13
14 >>> m.i = 4
15
16 >>> MyClass.i, m.i
17
18 (3, 4)
```

This is different from C++ and Java, but not so different from C#, where a static member can't be accessed using a reference to an instance.

45) Why do Python classes inherit object?

Why does the following class declaration inherit from object?

```
class MyClass(object):  
...
```

Answer#

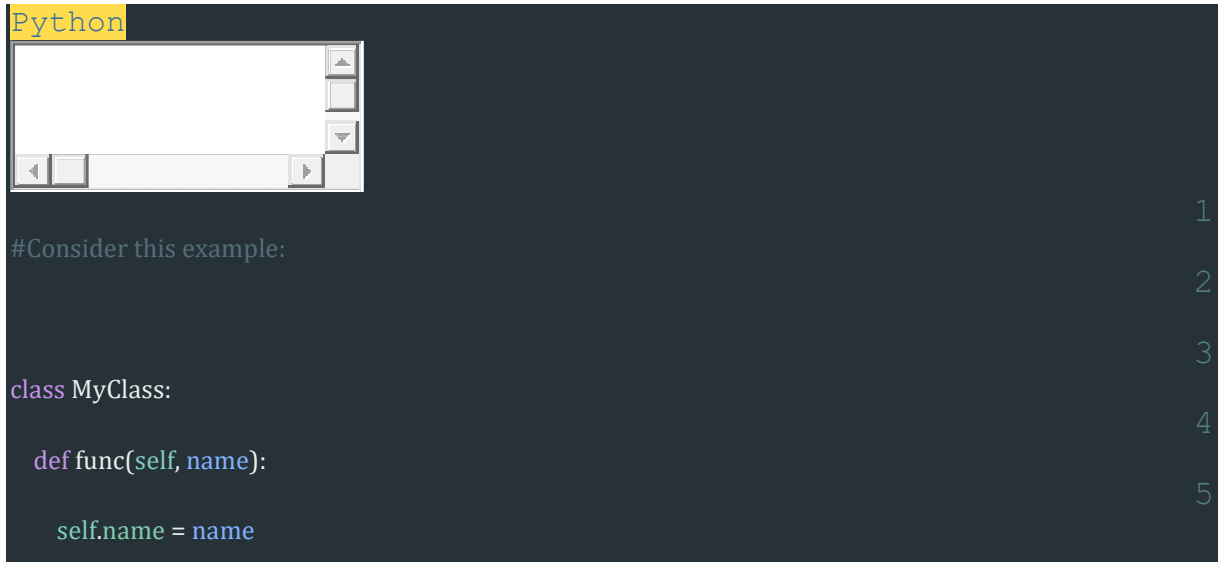
A screenshot of a code editor with a dark background. The editor has a tab labeled 'Python' in yellow. On the right side, there are line numbers from 1 to 9. The code is as follows:
Line 1: #Python 3
Line 2:
Line 3: class MyClass(object): = New-style class
Line 4:
Line 5: class MyClass: = New-style class (implicitly inherits from object)
Line 6:
Line 7: #Python 2
Line 8:
Line 9: class MyClass(object): = New-style class
Line 10: class MyClass: = OLD-STYLE CLASS

Explanation:

When defining base classes in Python 3.x, you're allowed to drop the object from the definition. However, this can open the door for a seriously hard-to-track problem...

Python introduced new-style classes back in Python 2.2, and by now old-style classes are really quite old.

46) What is the purpose of the 'self' parameter in Python? Why is it needed?



```
Python
#Consider this example:

class MyClass:

    def func(self, name):

        self.name = name
```

The screenshot shows a code editor window titled "Python". The code inside is a class definition for "MyClass" with a method "func" that takes "self" and "name" as parameters. The method body sets "self.name = name". To the right of the code, there is a vertical line number column with numbers 1 through 5 corresponding to the lines of code.

Answer#

The reason you need to use self. is because Python does not use a special syntax to refer to instance attributes.

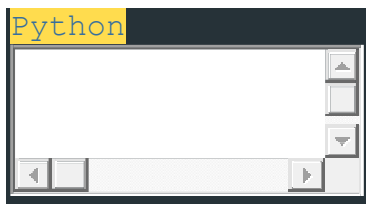
Python decided to do methods in a way that makes the instance to which the method belongs be passed automatically, but not received automatically: the first parameter of methods is the instance the method is called on.

That makes methods entirely the same as functions and leaves the actual name to use up to you (although the self is the convention, and people will generally frown at you when you use something else.) self is not special to the code, it's just another object.

Python could have done something else to distinguish normal names from attributes — special syntax like Ruby has, or requiring declarations like C++ and Java do, or perhaps something yet more different — but it didn't.

Python's all for making things explicit, making it obvious what's what, and although it doesn't do it entirely everywhere, it does do it for instance attributes.

That's why assigning to an instance attribute needs to know what instance to assign to, and that's why it needs self..

A small window titled 'Python' with a white background and a grey border, containing a text editor with a few lines of code.

```
1 #Example: Let's say you have a class ClassA which contains a method methodA defined as:
2
3
4 def methodA(self, arg1, arg2):
5     # do something
6
7 #and ObjectA is an instance of this class.
8
9
10 #Now when ObjectA.methodA(arg1, arg2) is called, python internally converts it for you as:
11
12 ClassA.methodA(ObjectA, arg1, arg2)
13
14 #The self variable refers to the object itself.
```

47) What is the difference between old-style and new-style classes in Python?

Answer#

Up to Python 2.1, old-style classes were the only flavor available to the user.

The concept of the (old-style) class is unrelated to the concept of type: if x is an instance of an old-style class, then x.**class** designates the class of x, but type(x) is always .

This reflects the fact that all old-style instances, independently of their class, are implemented with a single built-in type, called an instance.

New-style classes were introduced in Python 2.2 to unify the concepts of class and type. A new-style class is simply a user-defined type, no more, no less.

If `x` is an instance of a new-style class, then `type(x)` is typically the same as `x.class` (although this is not guaranteed – a new-style class instance is permitted to override the value returned for `x.class`).

The major motivation for introducing new-style classes is to provide a unified object model with a full meta-model.

It also has a number of immediate benefits, like the ability to subclass most built-in types, or the introduction of “descriptors”, which enable computed properties.

For compatibility reasons, classes are still old-style by default.

New-style classes are created by specifying another new-style class (i.e. a type) as a parent class, or the “top-level type” object if no other parent is needed.

The behavior of new-style classes differs from that of old-style classes in a number of important details in addition to what `type` returns.

Some of these changes are fundamental to the new object model, like the way special methods are invoked. Others are “fixes” that could not be implemented before for compatibility concerns, like the method resolution order in case of multiple inheritances.

Python 3 only has new-style classes.

No matter if you subclass from the object or not, classes are new-style in Python 3.

Declaration-wise:

New-style classes inherit from an object, or from another new-style class.

```
class NewStyleClass(object):  
    pass
```

```
class AnotherNewStyleClass(NewStyleClass):  
    pass
```

Old-style classes don't.

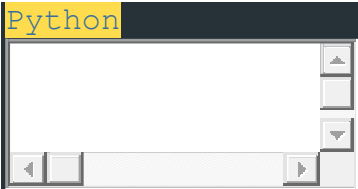
```
class OldStyleClass():  
    pass
```

Python 3 Note:

Python 3 doesn't support old-style classes, so either form noted above results in a new-style class.

48) How to call a parent class's method from a child class in Python?

Answer#



```
#Use the super() function:  
  
class Foo(Bar):  
    def baz(self, **kwargs):  
        return super().baz(**kwargs)  
  
#For Python < 3, you must explicitly opt in to using new-style classes and use:  
  
class Foo(Bar):  
    def baz(self, arg):  
        return super(Foo, self).baz(arg)
```

Example: `super(type[, object-or-type])`

Return a proxy object that delegates method calls to a parent or sibling class of type. This is useful for accessing inherited methods that have been overridden in a class.

The search order is the same as that used by `getattr()` except that the type itself is skipped.



```
Python
class A(object): # deriving from 'object' declares A as a 'new-style-class'
    def foo(self):
        print "foo"

class B(A):
    def foo(self):
        super(B, self).foo() # calls 'A.foo()'

myB = B()
myB.foo()
```

49) How to print instances of a class using `print()` in Python?

When I try to print an instance of a class, I get an output like this:



```
Python
>>> class Test():
...     def __init__(self):
```

```

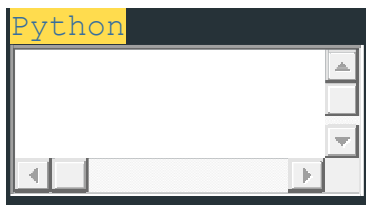
...     self.a = 'foo'
...
>>> print(Test())
<__main__.Test object at 0x7fc9a9e36d60>

```

How can I define the printing behavior (or the string representation) of a class and its instances?

For example, referring to the above code, how can I modify the Test class so that printing an instance shows the value?

Answer#



```

>>> class Test:
...     def __repr__(self):
...         return "Test()"
...     def __str__(self):
...         return "member of Test"
...
>>> t = Test()
>>> t
Test()
>>> print(t)
member of Test

```

The **`__str__`** method is what gets called happens when you print it, and the **`__repr__`** method is what happens when you use the `repr()` function (or when you look at it with the interactive prompt).

If no `__str__` method is given, Python will print the result of `__repr__` instead. If you define `__str__` but not `__repr__`, Python will use what you see above as the `__repr__`, but still use `__str__` for printing.

50) What is the difference between `__init__` and `__call__`?

I want to know the difference between `__init__` and `__call__` methods.



```
Python
#For example:

class test:

    def __init__(self):

        self.a = 10

    def __call__(self):

        b = 20
```

The image shows a Python IDE window with a dark background. The title bar says 'Python'. The code editor contains the following code: `#For example:`, `class test:`, `def __init__(self):`, `self.a = 10`, `def __call__(self):`, and `b = 20`. On the right side of the editor, there is a vertical line number column with numbers 1 through 10.

Answer#

So, the `__init__` method is used when the class is called to initialize the instance, while the `__call__` method is called when the instance is called.

Example:

The `__init__` is used to initialize newly created object, and receives arguments used to do that:

```
Python
class Foo:
    def __init__(self, a, b, c):
        # ...

x = Foo(1, 2, 3) # __init__

#The __call__ implements function call operator.

class Foo:
    def __call__(self, a, b, c):
        # ...

x = Foo()

x(1, 2, 3) # __call__
```

Python OOP Coding Interview Questions

51) What are meta classes in Python?

Answer#

A metaclass is the class of a class. A class defines how an instance of the class (i.e. an object) behaves while a metaclass defines how a class behaves. A class is an instance of a metaclass.

While in Python you can use arbitrary callables for metaclasses (like Jerub shows), the better approach is to make it an actual class itself. `type` is the usual metaclass in Python. `type` is itself a class, and it is its own type. You won't be able to recreate something like `type` purely in Python, but Python cheats a little. To create your own metaclass in Python you really just want to subclass `type`.

A metaclass is most commonly used as a class-factory. When you create an object by calling the class, Python creates a new class (when it executes the 'class' statement) by calling the metaclass.

Combined with the normal `__init__` and `__new__` methods, metaclasses, therefore, allow you to do 'extra things' when creating a class, like registering the new class with some registry or replacing the class with something else entirely.

When the class statement is executed, Python first executes the body of the class statement as a normal block of code. The resulting namespace (a dict) holds the attributes of the class-to-be.

The metaclass is determined by looking at the base classes of the class-to-be (metaclasses are inherited), at the **metaclass** attribute of the class-to-be (if any) or the `__metaclass__` global variable. The metaclass is then called with the name, bases and attributes of the class to instantiate it.

However, metaclasses actually define the type of a class, not just a factory for it, so you can do much more with them. You can, for instance, define normal methods on the metaclass.

These metaclass-methods are like class methods in that they can be called on the class without an instance, but they are also not like class methods in that they cannot be called on an instance of the class. `type.__subclasses__()` is an example of a method on the `type` metaclass.

You can also define the normal 'magic' methods, like `__add__`, `__iter__`, and `__getattr__`, to implement or change how the class behaves.

The `__metaclass__` attribute

In Python 2, you can add a `__metaclass__` attribute when you write a class (see next section for the Python 3 syntax):

```
class Foo(object):  
    __metaclass__ = something...  
[...]
```

If you do so, Python will use the metaclass to create the class Foo.

Metaclasses in Python 3

The syntax to set the metaclass has been changed in Python 3:

```
class Foo(object, metaclass=something):  
    ...
```

i.e. the **metaclass** attribute is no longer used, in favor of a keyword argument in the list of base classes.

The behavior of metaclasses however stays largely the same.

One thing added to metaclasses in Python 3 is that you can also pass attributes as keyword-arguments into a metaclass, like so:

```
class Foo(object, metaclass=something, kwarg1=value1, kwarg2=value2):
```

52) What is the difference between @staticmethod and @classmethod in Python?

Answer#

A `staticmethod` is a method that knows nothing about the class or instance it was called on. It just gets the arguments that were passed, no implicit first argument. It is basically useless in Python — you can just use a module function instead of a `staticmethod`.

A `classmethod`, on the other hand, is a method that gets passed the class it was called on, or the class of the instance it was called on, as first argument.

This is useful when you want the method to be a factory for the class: since it gets the actual class it was called on as first argument, you can always instantiate the right class, even when subclasses are involved.

Observe for instance how `dict.fromkeys()`, a classmethod, returns an instance of the subclass when called on a subclass:

A screenshot of a Python REPL window with a dark background. The title bar says 'Python'. The code is as follows:

```
>>> class DictSubclass(dict):  
...     def __repr__(self):  
...         return "DictSubclass"  
...  
>>> dict.fromkeys("abc")  
{'a': None, 'c': None, 'b': None}  
>>> DictSubclass.fromkeys("abc")  
DictSubclass  
>>>
```

Line numbers 1 through 10 are visible on the right side of the code block.

53) What is the meaning of single and double underscore before an object name in Python?

Answer#

Single Underscore – In a class, names with a leading underscore indicate to other programmers that the attribute or method is intended to be used inside that class.

However, privacy is not enforced in any way. Using leading underscores for functions in a module indicates it should not be imported from somewhere else.

From the PEP-8 style guide:

`_single_leading_underscore`: weak “internal use” indicator. E.g. `from M import *` does not import objects whose name starts with an underscore.

Double Underscore (Name Mangling)

From the Python docs:

Any identifier of the form `__spam` (at least two leading underscores, at most one trailing underscore) is textually replaced with `_classname__spam`, where `classname` is the current class name with leading underscore(s) stripped.

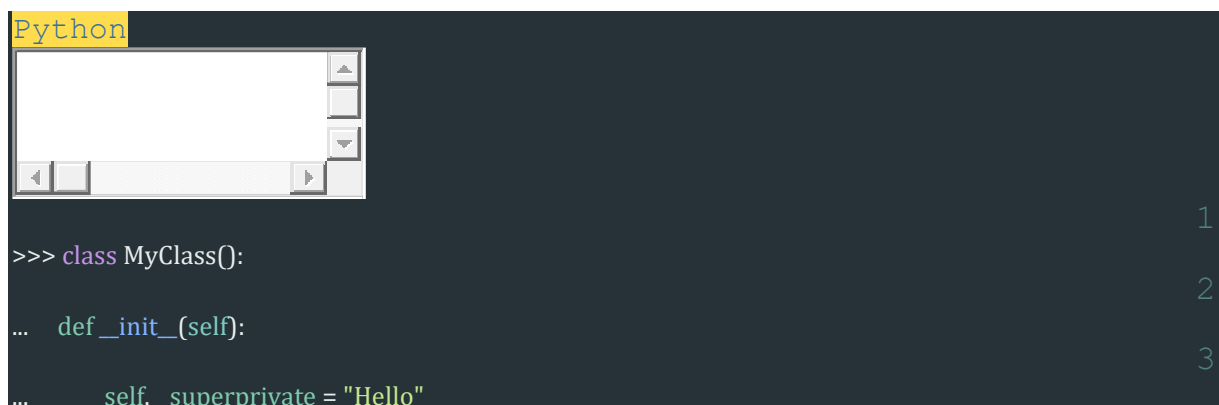
This mangling is done without regard to the syntactic position of the identifier, so it can be used to define class-private instance and class variables, methods, variables stored in globals, and even variables stored in instances. private to this class on instances of other classes.

And a warning from the same page:

Name mangling is intended to give classes an easy way to define “private” instance variables and methods, without having to worry about instance variables defined by derived classes, or mucking with instance variables by code outside the class.

Note that the mangling rules are designed mostly to avoid accidents; it still is possible for a determined soul to access or modify a variable that is considered private.

Example:

A screenshot of a Python REPL window. The title bar says "Python". The code being entered is a class definition. Line 1: `>>> class MyClass():`. Line 2: `... def __init__(self):`. Line 3: `... self.__superprivate = "Hello"`. The line numbers 1, 2, and 3 are visible on the right side of the code block.

```
>>> class MyClass():
...     def __init__(self):
...         self.__superprivate = "Hello"
```

```

...     self._semiprivate = ", world!"
...
>>> mc = MyClass()
>>> print mc.__superprivate
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: myClass instance has no attribute '__superprivate'
>>> print mc._semiprivate
, world!
>>> print mc.__dict__
{'_MyClass__superprivate': 'Hello', '_semiprivate': ', world!'}
```

54) What are the differences between type() and isinstance() in Python?

Answer#

isinstance caters for inheritance (an instance of a derived class is an instance of a base class, too), while checking for equality of type does not (it demands identity of types and rejects instances of subtypes, AKA subclasses).

Normally, in Python, you want your code to support inheritance, of course (since inheritance is so handy, it would be bad to stop code using yours from using it!), so isinstance is less bad than checking identity of types because it seamlessly supports inheritance.

Python



```
#Using type:
```

```
import types
```

```
if type(a) is types.DictType:
```

```
    do_something()
```

```
if type(b) in types.StringTypes:
```

```
    do_something_else()
```

```
#Using isinstance:
```

```
if isinstance(a, dict):
```

```
    do_something()
```

```
if isinstance(b, str) or isinstance(b, unicode):
```

```
    do_something_else()
```

55) What is a mixin and why is it useful in Python?

Answer#

A mixin is a special kind of multiple inheritance. There are two main situations where mixins are used:

- 1) You want to provide a lot of optional features for a class.
- 2) You want to use one particular feature in a lot of different classes.

For an example of number one, consider werkzeug's request and response system. I can make a plain old request object by saying:

```
Python
1
2
3
4
5
from werkzeug import BaseRequest

class Request(BaseRequest):

    pass
```

If I want to add accept header support, I would make that

```
Python
1
2
3
4
from werkzeug import BaseRequest, AcceptMixin

class Request(AcceptMixin, BaseRequest):

    pass
```

If I wanted to make a request object that supports accept headers, etags, authentication, and user agent support, I could do this:

```
Python
1
2
3
4
from werkzeug import BaseRequest, AcceptMixin, ETagRequestMixin, UserAgentMixin,
AuthenticationMixin

class Request(AcceptMixin, ETagRequestMixin, UserAgentMixin, AuthenticationMixin, BaseRequest):

    pass
```

The difference is subtle, but in the above examples, the mixin classes weren't made to stand on their own. In more traditional multiple inheritance, the AuthenticationMixin (for example) would probably be something more like Authenticator. That is, the class would probably be designed to stand on its own.

56) What is the purpose of `__slots__` in Python?

Answer#

The special attribute `__slots__` allows you to explicitly state which instance attributes you expect your object instances to have, with the expected results:

- 1) faster attribute access.
- 2) space savings in memory.

The space savings is from

- 1) Storing value references in slots instead of `__dict__`.
- 2) Denying `__dict__` and `__weakref__` creation if parent classes deny them and you declare `__slots__`.

Quick Caveats

A small caveat, you should only declare a particular slot one time in an inheritance tree.

For example:

A screenshot of a Python IDE with a dark background. The word 'Python' is in the top left corner. A code editor window shows the following code:

```
class Base:
    __slots__ = 'foo', 'bar'
```

Line numbers 1, 2, and 3 are visible on the right side of the editor.

```

class Right(Base):
    __slots__ = 'baz',

class Wrong(Base):
    __slots__ = 'foo', 'bar', 'baz'    # redundant foo and bar

```

Python doesn't object when you get this wrong (it probably should), problems might not otherwise manifest, but your objects will take up more space than they otherwise should. Python 3.8:

```

Python
>>> from sys import getsizeof
>>> getsizeof(Right()), getsizeof(Wrong())
(56, 72)

#This is because the Base's slot descriptor has a slot separate from the Wrong's. This shouldn't usually
come up, but it could:

>>> w = Wrong()
>>> w.foo = 'foo'
>>> Base.foo.__get__(w)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: foo

>>> Wrong.foo.__get__(w)
'foo'

```

The biggest caveat is for multiple inheritance – multiple “parent classes with nonempty slots” cannot be combined.

To accommodate this restriction, follow best practices: Factor out all but one or all parents’ abstraction which their concrete class respectively and your new concrete class collectively will inherit from – giving the abstraction(s) empty slots (just like abstract base classes in the standard library).

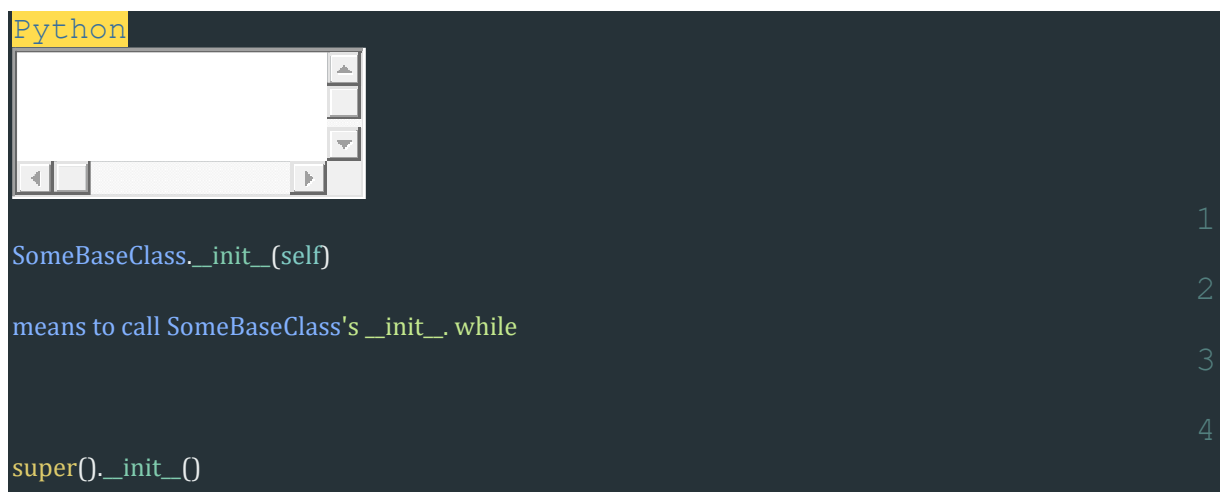
57) What does ‘super’ do in Python? What is the difference between `super().__init__()` and explicit superclass `__init__()`?

Answer#

The benefits of `super()` in single-inheritance are minimal — mostly, you don’t have to hard-code the name of the base class into every method that uses its parent methods.

However, it’s almost impossible to use multiple-inheritance without `super()`. This includes common idioms like mixins, interfaces, abstract classes, etc. This extends to code that later extends yours. If somebody later wanted to write a class that extended `Child` and a mixin, their code would not work properly.

What’s the difference?



```

Python
SomeBaseClass.__init__(self)
means to call SomeBaseClass's __init__. while
super().__init__()
  
```

means to call a bound **init** from the parent class that follows SomeBaseClass's child class (the one that defines this method) in the instance's Method Resolution Order (MRO).

If the instance is a subclass of this child class, there may be a different parent that comes next in the MRO.

Explained simply

When you write a class, you want other classes to be able to use it. `super()` makes it easier for other classes to use the class you're writing.

As Bob Martin says, the good architecture allows you to postpone decision-making as long as possible.

`super()` can enable that sort of architecture.

When another class subclasses the class you wrote, it could also be inheriting from other classes. And those classes could have an **__init__** that comes after this **__init__** based on the ordering of the classes for method resolution.

Without `super` you would likely hard-code the parent of the class you're writing (like the example does). This would mean that you would not call the next **__init__** in the MRO, and you would thus not get to reuse the code in it.

If you're writing your own code for personal use, you may not care about this distinction. But if you want others to use your code, using `super` is one thing that allows greater flexibility for users of the code.

Python 2 versus 3



```
#This works in Python 2 and 3:
super(Child, self).__init__()
```

The screenshot shows a dark-themed Python IDE window. The title bar is labeled 'Python'. The code editor contains two lines of text. The first line is a comment: `#This works in Python 2 and 3:`. The second line is a code statement: `super(Child, self).__init__()`. The text is color-coded: `super` is blue, `Child` is green, `self` is yellow, and `__init__()` is white. On the right side of the editor, there are three line numbers: 1, 2, and 3, corresponding to the lines of code.

```
#This only works in Python 3:
```

```
super().__init__()
```

It works with no arguments by moving up in the stack frame and getting the first argument to the method (usually self for an instance method or cls for a class method – but could be other names) and finding the class (e.g. Child) in the free variables (it is looked up with the name `__class__` as a free closure variable in the method).

58) Explain the '`__enter__`' and '`__exit__`' methods in Python?

Answer#

Using these magic methods (`__enter__`, `__exit__`) allows you to implement objects which can be used easily with the with statement.

The idea is that it makes it easy to build code which needs some 'cleandown' code executed (think of it as a try-finally block).

A useful example could be a database connection object (which then automatically closes the connection once the corresponding 'with'-statement goes out of scope):

```
Python
class DatabaseConnection(object):
```

```
    def __enter__(self):
```

```
        # make a database connection and return it
```

```

...
return self.dbconn

def __exit__(self, exc_type, exc_val, exc_tb):
    # make sure the dbconnection gets closed
    self.dbconn.close()
...

```

As explained above, use this object with the with statement (you may need to do from **future** import with_statement at the top of the file if you're on Python 2.5).

```
with DatabaseConnection() as mydbconn:
```

```
# do stuff
```

PEP343 — The 'with' statement' has a nice writeup as well.

59) How do I implement interfaces in Python?

Answer#

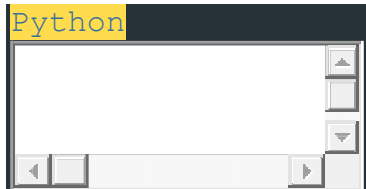
Interfaces are not necessary in Python. This is because Python has proper multiple inheritance, and also ducktyping, which means that the places where you must have interfaces in Java, you don't have to have them in Python.

That said, there are still several uses for interfaces. Some of them are covered by Python's Abstract Base Classes, introduced in Python 2.6. They are useful, if you want to make base classes that cannot be instantiated, but provide a specific interface or part of an implementation.

Another usage is if you somehow want to specify that an object implements a specific interface, and you can use ABC's for that too by subclassing from them. Another way is zope.interface, a module that is a part of the Zope Component Architecture, a really awesomely cool component framework.

Here you don't subclass from the interfaces, but instead mark classes (or even instances) as implementing an interface. This can also be used to look up components from a component registry. Supercool!

60) How to invoke the super constructor in Python?



```
Python
class A:
    def __init__(self):
        print("world")

class B(A):
    def __init__(self):
        print("hello")

B() # output: hello
```

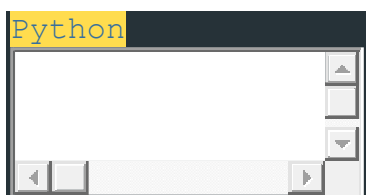
1
2
3
4
5
6
7
8
9

In all other languages I've worked with the super constructor is invoked implicitly. How does one invoke it in Python? I would expect `super(self)` but this doesn't work.

Answer#

there are multiple ways to call super class methods (including the constructor), however in Python-3.x the process has been simplified:

Python-3.x



```
Python
class A(object):
```

1
2

```

def __init__(self):
    print("world")

class B(A):
    def __init__(self):
        print("hello")
        super().__init__()

```

Python-2.x

In python 2.x, you have to call the slightly more verbose version `super(, self)`, which is equivalent to `super()` as per the docs.



```

class A(object):
    def __init__(self):
        print "world"

class B(A):
    def __init__(self):
        print "hello"
        super(B, self).__init__()

```

Python Functions Coding Interview Questions

61) How to get a function name as a string in Python?

```
Python
def foo():
    pass

>>> name_of(foo)

"foo"
```

Answer#

my_function.__name__

Using **__name__** is the preferred method as it applies uniformly. Unlike func_name, it works on built-in functions as well:

```
Python
>>> import time
>>> time.time.func_name

Traceback (most recent call last):
  File "<stdin>", line 1, in ?
AttributeError: 'builtin_function_or_method' object has no attribute 'func_name'

>>> time.time.__name__

'time'
```

Also, the double underscores indicate to the reader this is a special attribute. As a bonus, classes and modules have a `__name__` attribute too, so you only have to remember one special name.

62) What is the naming convention in Python for variables and functions?

Answer#

Function names should be lowercase, with words separated by underscores as necessary to improve readability.

Variable names follow the same convention as function names.

mixedCase is allowed only in contexts where that's already the prevailing style (e.g. `threading.py`), to retain backwards compatibility.

The Google Python Style Guide has the following convention:

module_name, package_name, ClassName, method_name, ExceptionName, function_name, GLOBAL_CONSTANT_NAME, global_var_name, instance_var_name, function_parameter_name, local_var_name.

A similar naming scheme should be applied to a CLASS_CONSTANT_NAME

63) How are Python lambdas useful?

Answer#

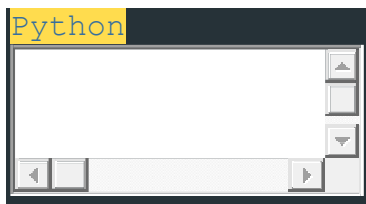
Lambda Expressions:

`lambda x: x*2 + 2x - 5`

Those things are actually quite useful. Python supports a style of programming called functional programming where you can pass functions to other functions to do stuff. Example:

```
mult3 = filter(lambda x: x % 3 == 0, [1, 2, 3, 4, 5, 6, 7, 8, 9])
```

sets `mult3` to `[3, 6, 9]`, those elements of the original list that are multiples of 3. This is shorter (and, one could argue, clearer) than



```
Python
def filterfunc(x):
    return x % 3 == 0
mult3 = filter(filterfunc, [1, 2, 3, 4, 5, 6, 7, 8, 9])
```

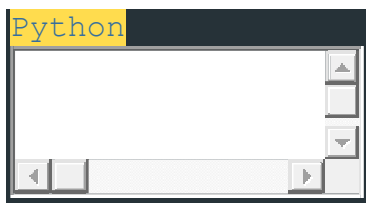
1
2
3

Of course, in this particular case, you could do the same thing as a list comprehension:

```
mult3 = [x for x in [1, 2, 3, 4, 5, 6, 7, 8, 9] if x % 3 == 0]
```

(or even as `range(3,10,3)`), but there are many other, more sophisticated use cases where you can't use a list comprehension and a lambda function may be the shortest way to write something out.

Returning a function from another function



```
Python
def transform(n):
    ... return lambda x: x + n
    ...
f = transform(3)
f(4)
7
```

1
2
3
4
5
6
7

This is often used to create function wrappers, such as Python's decorators.

Combining elements of an iterable sequence with `reduce()`

```
reduce(lambda a, b: '{} {}'.format(a, b), [1, 2, 3, 4, 5, 6, 7, 8, 9])  
'1, 2, 3, 4, 5, 6, 7, 8, 9'
```

Sorting by an alternate key

```
sorted([1, 2, 3, 4, 5, 6, 7, 8, 9], key=lambda x: abs(5-x))  
[5, 4, 6, 3, 7, 2, 8, 1, 9]
```

I use lambda functions on a regular basis. It took me a while to get used to them, but eventually, I came to understand that they're a very valuable part of the language.

lambda is just a fancy way of saying function. Other than its name, there is nothing obscure, intimidating, or cryptic about it. When you read the following line, replace lambda with function in your mind:

```
f = lambda x: x + 1  
f(3)  
4
```

It just defines a function of x. Some other languages, like R, say it explicitly:

```
f = function(x) { x + 1 }  
f(3)  
4
```

Do you see? It's one of the most natural things to do in programming.

64) How do I call a function from another .py file?

file.py contains a function named function. How do I import it?

```
from file.py import function(a,b)
```

The above gives an error:

```
ImportError: No module named 'file.py'; file is not a package
```

Answer#

First, import function from file.py:

from the file import function

Later, call the function using:

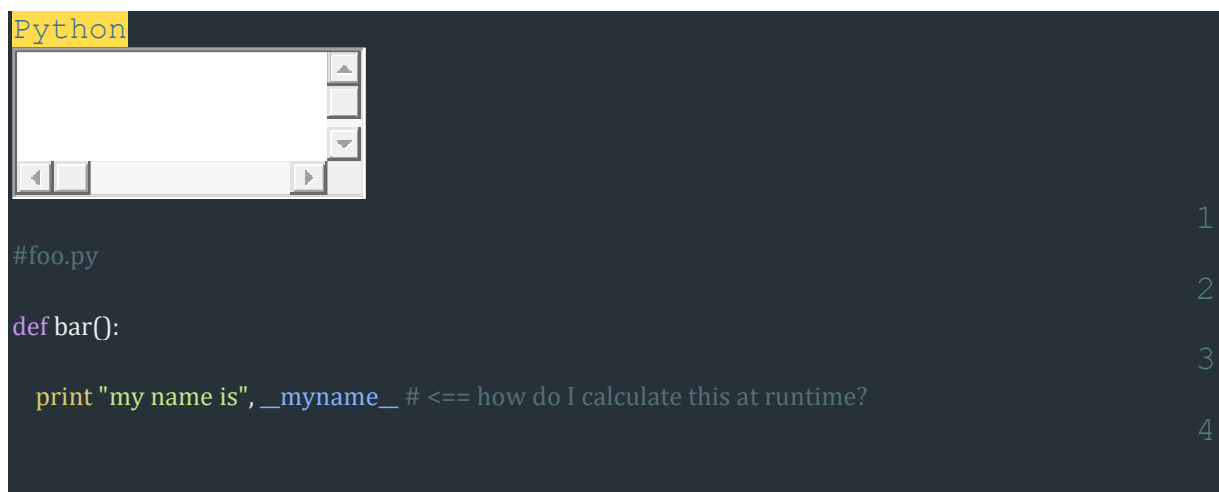
function(a, b)

Note that if you're trying to import functions from a.py to a file called b.py, you will need to make sure that a.py and b.py are in the same directory.

65) How to determine function name from within that function (without using traceback) in Python?

In Python, without using the traceback module, is there a way to determine a function's name from within that function?

Say I have a module foo with a function bar. When executing foo.bar(), is there a way for bar to know bar's name? Or better yet, foo.bar's name?

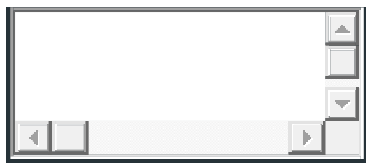
A screenshot of a Python IDE with a dark theme. The title bar at the top left says "Python" in yellow. The code editor shows a file named "#foo.py" on the first line. The second line is "def bar:". The third line is "print 'my name is', __myname__ # <== how do I calculate this at runtime?". The line numbers 1, 2, 3, and 4 are visible on the right side of the editor.

```
Python
#foo.py
def bar():
    print "my name is", __myname__ # <== how do I calculate this at runtime?
```

Answer#

A screenshot of a Python IDE title bar. The title bar is dark grey with the word "Python" in yellow text on the left side.

```
Python
```



```
import inspect

def foo():

    print(inspect.stack()[0][3])

    print(inspect.stack()[1][3]) # will give the caller of foos name, if something called foo

foo()

#output:

foo

<module_caller_of_foo>
```

66) Why do some functions have underscores “__” before and after the function name in Python?

Answer#

Descriptive: Naming Styles in Python

The following special forms using leading or trailing underscores are recognized (these can generally be combined with any case convention):

`_single_leading_underscore`: weak “internal use” indicator. E.g. `from M import *` does not import objects whose name starts with an underscore.

`single_trailing_underscore_`: used by convention to avoid conflicts with Python keyword, e.g.

`Tkinter.Toplevel(master, class_='ClassName')`

`__double_leading_underscore`: when naming a class attribute, invokes name mangling (inside class `FooBar`, `__boo` becomes `_FooBar__boo`; see below).

`__double_leading_and_trailing_underscore__`: "magic" objects or attributes that live in user-controlled namespaces. E.g. **init**, **import** or **file**. Never invent such names; only use them as documented.

Note that names with double leading and trailing underscores are essentially reserved for Python itself: "Never invent such names; only use them as documented".

67) How to run a function from the command line in Python?

I have this code:

```
def hello():  
    return 'Hi :)'
```

How would I run this directly from the command line?

Answer#

With the `-c` (command) argument (assuming your file is named `foo.py`):

```
$ python -c 'import foo; print foo.hello()'
```

Alternatively, if you don't care about namespace pollution:

```
$ python -c 'from foo import *; print hello()'
```

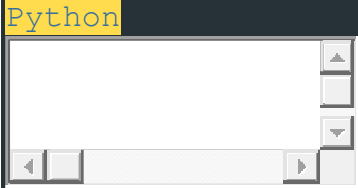
And the middle ground:

```
$ python -c 'from foo import hello; print hello()'
```

68) How to pass a dictionary to a function as keyword parameters in Python?

I'd like to call a function in python using a dictionary with matching key-value pairs for the parameters.

Here is some code:

A small window titled "Python" with a text editor and standard window controls (minimize, maximize, close).

```
d = dict(param='test')  
  
def f(param):  
    print(param)  
  
f(d)  
  
#This prints {'param': 'test'} but I'd like it to just print test.  
  
#I'd like it to work similarly for more parameters:  
  
d = dict(p1=1, p2=2)  
  
def f2(p1, p2):  
    print(p1, p2)  
  
f2(d)  
  
#Is this possible?
```

Answer#

Just include the `**` operator to unpack the dictionary. This solves the issue of passing a dictionary to a function as a keyword parameter.

So here is an example:

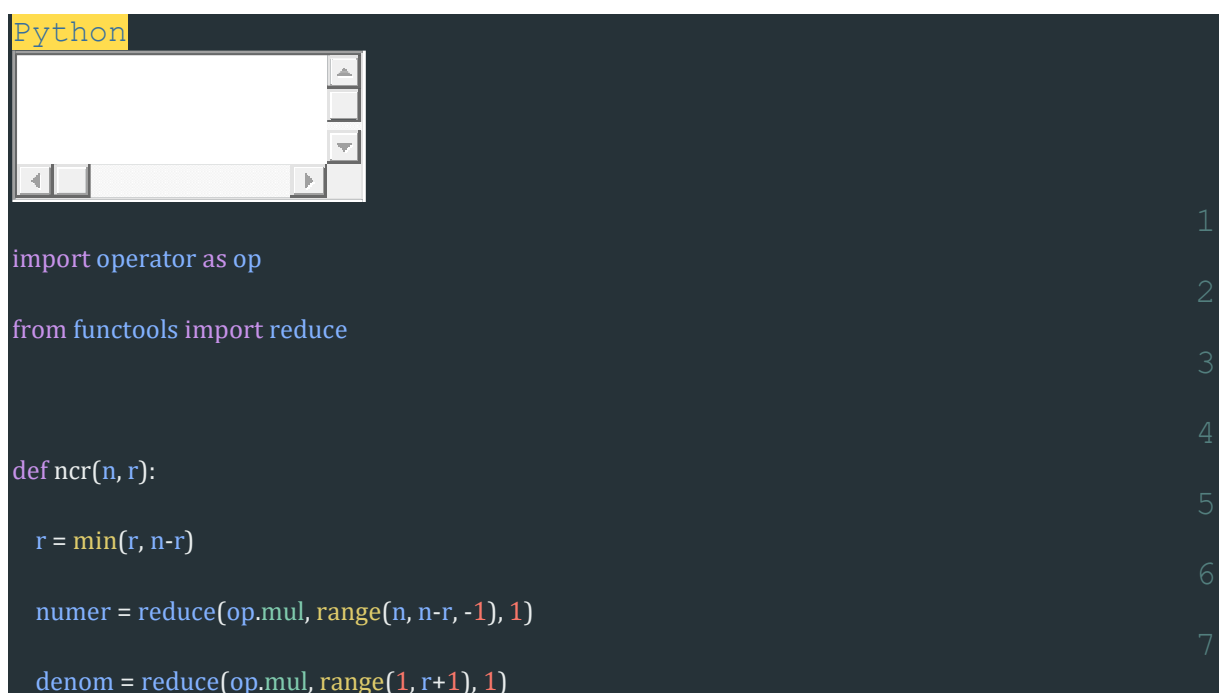


```
Python
1 d = dict(p1=1, p2=2)
2
3 def f2(p1,p2):
4     print p1, p2
f2(**d)
```

69) Is there a math nCr function in Python?

Answer# Python code for nCr (n Choose r) function:

The following program calculates nCr in an efficient manner (compared to calculating factorials etc.)



```
Python
1 import operator as op
2
3 from functools import reduce
4
5 def ncr(n, r):
6     r = min(r, n-r)
7     numer = reduce(op.mul, range(n, n-r, -1), 1)
8     denom = reduce(op.mul, range(1, r+1), 1)
```

```

return numer // denom # or / in Python 2
#As of Python 3.8, binomial coefficients are available in the standard library as math.comb:
>>> from math import comb
>>> comb(10,3)
120

```

70) How to send an email with Python? Python code for sending an email?

Answer#: Python program for sending an email.

We recommend using the standard packages email and smtplib together to send emails.

Please look at the following example (reproduced from the Python documentation).

Notice that if you follow this approach, the “simple” task is indeed simple, and the more complex tasks (like attaching binary objects or sending plain/HTML multipart messages) are accomplished very rapidly.

```

Python
# Import smtplib for the actual sending function
import smtplib

# Import the email modules we'll need

```

```

from email.mime.text import MIMEText
6
7
# Open a plain text file for reading. For this example, assume that
8
# the text file contains only ASCII characters.
9
with open(textfile, 'rb') as fp:
10
    # Create a text/plain message
11
    msg = MIMEText(fp.read())
12
13
# me == the sender's email address
14
# you == the recipient's email address
15
msg['Subject'] = 'The contents of %s' % textfile
16
msg['From'] = me
17
msg['To'] = you
18
19
# Send the message via our own SMTP server, but don't include the
20
# envelope header.
21
s = smtplib.SMTP('localhost')
22
s.sendmail(me, [you], msg.as_string())
23
s.quit()
24

```

For sending emails to multiple destinations, you can follow the below Python example:

```

Python
1
# Import smtplib for the actual sending function
2

```

```
import smtplib 3

# Here are the email package modules we'll need 4

from email.mime.image import MIMEImage 5

from email.mime.multipart import MIMEMultipart 6

# Create the container (outer) email message. 7

msg = MIMEMultipart() 8

msg['Subject'] = 'Our family reunion' 9

# me == the sender's email address 10

# family = the list of all recipients' email addresses 11

msg['From'] = me 12

msg['To'] = ', '.join(family) 13

msg.preamble = 'Our family reunion' 14

# Assume we know that the image files are all in PNG format 15

for file in pngfiles: 16

    # Open the files in binary mode. Let the MIMEImage class automatically 17

    # guess the specific image type. 18

    with open(file, 'rb') as fp: 19

        img = MIMEImage(fp.read()) 20

        msg.attach(img) 21

# Send the email via our own SMTP server. 22

s = smtplib.SMTP('localhost') 23

s.sendmail(me, family, msg.as_string()) 24

25
26
27
28
```

```
s.quit()
```

As you can see, the header To in the MIMEText object must be a string consisting of email addresses separated by commas. On the other hand, the second argument to the sendmail function must be a list of strings (each string is an email address).

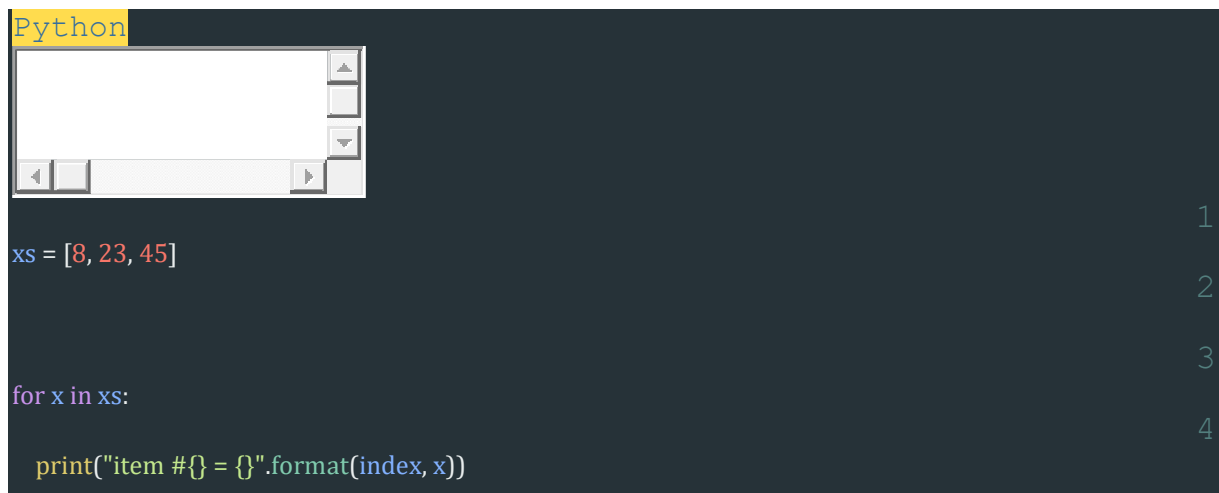
So, if you have three email addresses: person1@example.com, person2@example.com, and person3@example.com, you can do as follows (obvious sections omitted):

```
to = ["person1@example.com", "person2@example.com",  
      "person3@example.com"]  
msg['To'] = ",".join(to)  
s.sendmail(me, to, msg.as_string())
```

the `",".join(to)` part makes a single string out of the list, separated by commas.

Python List/Loops Coding Interview Questions & Answers

71) What is the output of the below Python lists program?



```
Python  
xs = [8, 23, 45]  
  
for x in xs:  
    print("item #{} = {}".format(index, x))
```

Answer#:

NameError: name 'index' is not defined

72) How do I access the index while iterating over a sequence with a for loop in Python?

Answer#:

```
Python
xs = [8, 23, 45]
for idx, x in enumerate(xs):
    print(idx, x)
```

Output:

```
0 8
1 23
2 45
```

73) How to traverse a list in reverse order in Python?

Answer#:

```
Python
#Use the built-in reversed() function:

>>> a = ["foo", "bar", "baz"]
>>> for i in reversed(a):
...     print(i)
```

```

...
baz
bar
foo

#To also access the original index, use enumerate() on your list before passing it to reversed():

>>> for i, e in reversed(list(enumerate(a))):
...     print(i, e)
...
2 baz
1 bar
0 foo

```

Since `enumerate()` returns a generator and generators can't be reversed, you need to convert it to a list first.

74) What is the difference between range and xrange functions in Python?

Answer#:

In Python 2.x:

`range` creates a list, so if you do `range(1, 10000000)` it creates a list in memory with 9999999 elements.

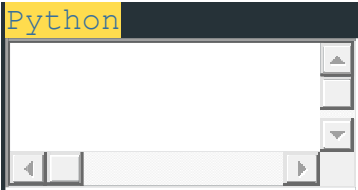
`xrange` is a sequence object that evaluates lazily.

In Python 3:

range does the equivalent of Python 2's xrange. To get the list, you have to explicitly use list(range(...)).
xrange no longer exists.

75) How can I break out of multiple loops/nested-loops?

Is there an easier way to break out of nested loops than throwing an exception?



```
1 for x in range(10):
2
3     for y in range(10):
4
5         print x*y
6
7         if x*y > 50:
8
9             "break both loops"
10
11
12
13
14
15
16
```

#I.e., is there a nicer way than:

```
class BreakIt(Exception): pass

try:

    for x in range(10):

        for y in range(10):

            print x*y

            if x*y > 50:
```

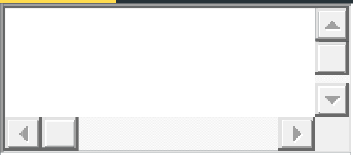
```

        raise BreakIt
    except BreakIt:
        pass

```

Answer# Here is the code for breaking the nested loops

Python



```

1  for x in xrange(10):
2
3      for y in xrange(10):
4
5          print x*y
6
7          if x*y > 50:
8
9              break
10
11         else:
12
13             continue # only executed if the inner loop did NOT break
14
15         break # only executed if the inner loop DID break
16
17 #The same works for deeper loops:
18
19 for x in xrange(10):
20
21     for y in xrange(10):
22
23         for z in xrange(10):
24
25             print x,y,z
26
27             if x*y*z == 30:
28
29                 break
30
31         else:

```

```
continue 19
break 20
else: 21
    continue 22
break 23
```

76) How to loop backward using indices in Python?

I am trying to loop from 100 to 0. How do I do this in Python?

for i in range (100,0), It doesn't work. How to solve this?

Answer# 1

Try range(100,-1,-1), the 3rd argument being the increment to use.

Answer# 2

```
for i in reversed(xrange(101)):
    print i,
```

77) Is there any way in Python to combine for-loop and if-statement?

Answer#

Using generator expressions we can combine for-loop and if-statements in Python:



```
gen = (x for x in xyz if x not in a)
```

2

```
for x in gen:
```

3

```
    print(x)
```

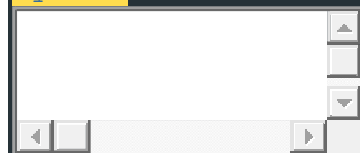
4

78) How do I loop through a list by two items at a time in Python?

Answer#

You can use a range with a step size of 2:

Python



```
#Python 2
```

1

2

3

```
for i in xrange(0,10,2):
```

4

```
    print(i)
```

5

6

7

```
#Python 3
```

8

9

```
for i in range(0,10,2):
```

10

```
    print(i)
```

Note: Use xrange in Python 2 instead of range because it is more efficient as it generates an iterable object, and not the whole list.

79) How do you create different variable names while in a loop in Python?

For example purposes...

```
for x in range(0,9):  
    string'x' = "Hello"
```

So I end up with string1, string2, string3... all equaling "Hello"

Answer#

Sure you can; it's called a dictionary:

A screenshot of a Python IDE window with a dark background. The window title is "Python" in a yellow tab. The code editor shows a dictionary being created and populated. Line 1: `d = {}`. Line 2: `for x in range(1, 10):`. Line 3: `d["string{0}".format(x)] = "Hello"`. Line 4: `>>> d["string5"]`. Line 5: `'Hello'`. Line 6: `>>> d`. Line 7: `{'string1': 'Hello',`. Line 8: `'string2': 'Hello',`. Line 9: `'string3': 'Hello',`. Line 10: `'string4': 'Hello',`. Line 11: `'string5': 'Hello',`. Line 12: `'string6': 'Hello',`. Line 13: `'string7': 'Hello',`. Line 14: `'string8': 'Hello',`. Line 15: `'string9': 'Hello'}`. Line 16: The code ends. The lines are numbered 1 through 16 on the right side of the editor.

```
Python  
  
d = {}  
  
for x in range(1, 10):  
    d["string{0}".format(x)] = "Hello"  
  
>>> d["string5"]  
'Hello'  
>>> d  
{'string1': 'Hello',  
'string2': 'Hello',  
'string3': 'Hello',  
'string4': 'Hello',  
'string5': 'Hello',  
'string6': 'Hello',  
'string7': 'Hello',  
'string8': 'Hello',  
'string9': 'Hello'}
```

I said this somewhat tongue in cheek, but really the best way to associate one value with another value is a dictionary. That is what it was designed for!

80) Is there a “do ... until” loop in Python?

Is there a

do until x:

...

in Python, or a nice way to implement such a looping construct?

Answer#

There is no do-while loop in Python.

This is a similar construct, taken from the link above.

A screenshot of a Python IDE with a dark background. The word "Python" is in a yellow box at the top left. Below it is a code editor window with a white background and a scrollbar. The code in the editor is:

```
while True:
    do_something()
    if condition():
        break
```

Line numbers 1, 2, 3, and 4 are visible on the right side of the code editor.

81) How to reverse tuples in Python?

Answer#

There are two idiomatic ways to do this:

`reversed(x)` # returns an iterator

or

`x[::-1]` # returns a new tuple

Since tuples are immutable, there is no way to reverse a tuple in-place.

The iterator returned by reversed would be equivalent to

A screenshot of a Python IDE with a dark background. The word "Python" is in a yellow tab at the top left. A code editor window is open, showing a function definition. To the right of the code, line numbers 1, 2, and 3 are visible. The code defines a generator function named myreversed that takes a sequence 'seq' as an argument. It uses a for loop with 'i' in 'range(len(x) - 1, -1, -1)' to iterate from the end of the sequence to the beginning, yielding 'seq[i]' at each step.

```
def myreversed(seq):  
    for i in range(len(x) - 1, -1, -1):  
        yield seq[i]
```

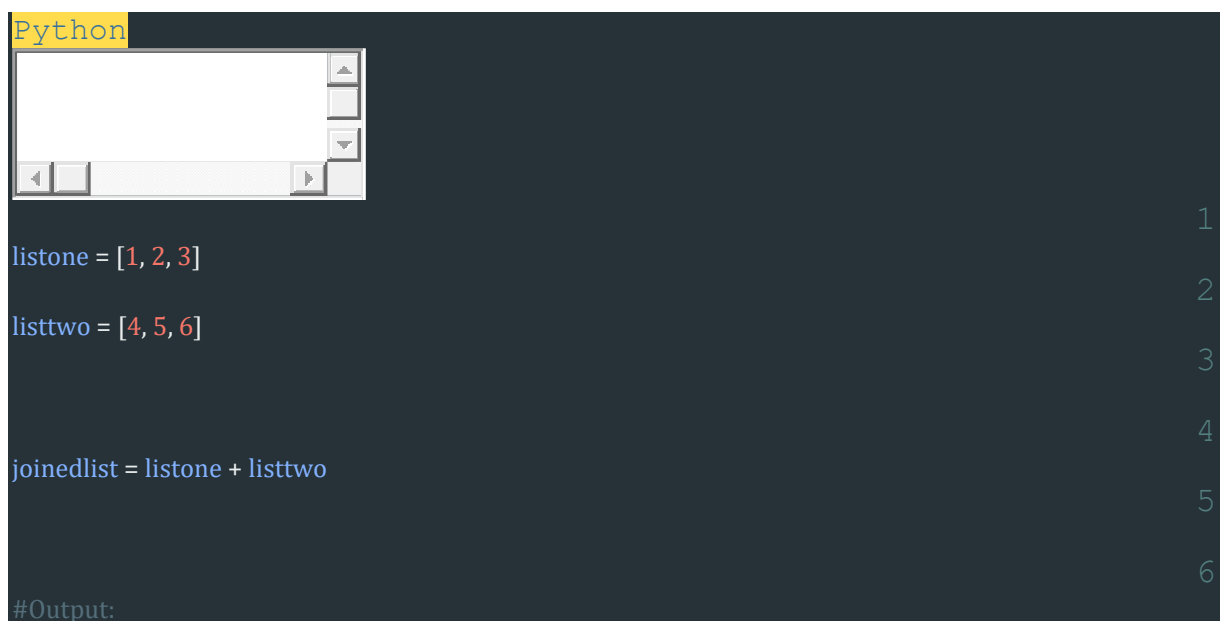
i.e. it relies on the sequence having a known length to avoid having to actually reverse the tuple.

As to which is more efficient, I'd suspect it'd be the `seq[::-1]` if you are using all of it and the tuple is small and reversed when the tuple is large, but performance in python is often surprising so measure it!

82) How do I concatenate two lists in Python?

Answer#

Use the + operator to combine the lists:

A screenshot of a Python IDE with a dark background. The word "Python" is in a yellow tab at the top left. A code editor window is open, showing two lists being defined and then concatenated. To the right of the code, line numbers 1 through 6 are visible. The code defines 'listone' as [1, 2, 3] and 'listtwo' as [4, 5, 6]. It then creates 'joinedlist' by adding 'listone' and 'listtwo'. At the bottom, there is a comment '#Output:'.

```
listone = [1, 2, 3]  
listtwo = [4, 5, 6]  
  
joinedlist = listone + listtwo  
  
#Output:
```

```
>>> joinedlist
```

```
[1, 2, 3, 4, 5, 6]
```

83) How do I check if a list is empty?

For example, if passed the following:

```
a = []
```

How do I check to see if a is empty?

Answer#

```
if not a:  
    print("List is empty")
```

Using the implicit booleanness of the empty list is quite Pythonic.

84) What's the difference between the Python list methods `append()` and `extend()`?

Answer#

`append` appends a specified object at the end of the list:

Python

```
>>> x = [1, 2, 3]
```

```
>>> x.append([4, 5])
```

```
>>> print(x)
```

```
[1, 2, 3, [4, 5]]
```

```
#extend extends the list by appending elements from the specified iterable:
```

```
>>> x = [1, 2, 3]
```

```
>>> x.extend([4, 5])
```

```
>>> print(x)
```

```
[1, 2, 3, 4, 5]
```

85) How do I get the last element of a list in Python?

Answer#

`some_list[-1]` is the shortest and most Pythonic.

In fact, you can do much more with this syntax. The `some_list[-n]` syntax gets the *n*th-to-last element. So `some_list[-1]` gets the last element, `some_list[-2]` gets the second to last, etc, all the way down to `some_list[-len(some_list)]`, which gives you the first element.

You can also set list elements in this way. For instance:

Python

```
>>> some_list = [1, 2, 3]
```

```
>>> some_list[-1] = 5 # Set the last element
```

```
>>> some_list[-2] = 3 # Set the second to last element
```

```
>>> some_list
```

```
[1, 3, 5]
```

Note that getting a list item by index will raise an `IndexError` if the expected item doesn't exist. This means that `some_list[-1]` will raise an exception if `some_list` is empty, because an empty list can't have a last element.

86) How do I sort a list of dictionaries by a value of the dictionary in Python?

How to sort a list of dictionaries by a specific key's value? Given:

```
[{'name': 'Homer', 'age': 39}, {'name': 'Bart', 'age': 10}]
```

When sorted by name, it should become:

```
[{'name': 'Bart', 'age': 10}, {'name': 'Homer', 'age': 39}]
```

Answer#

The `sorted()` function takes a `key=` parameter

```
newlist = sorted(list_to_be_sorted, key=lambda d: d['name'])
```

Alternatively, you can use `operator.itemgetter` instead of defining the function yourself

```
from operator import itemgetter
newlist = sorted(list_to_be_sorted, key=itemgetter('name'))
```

For completeness, add `reverse=True` to sort in descending order

```
newlist = sorted(list_to_be_sorted, key=itemgetter('name'), reverse=True)
```

87) How can I randomly select an item from a list?

How do I retrieve an item at random from the following list?

```
foo = ['a', 'b', 'c', 'd', 'e']
```

Answer#

```
Python
1
2
3
4
5
6
Use random.choice():

import random

foo = ['a', 'b', 'c', 'd', 'e']

print(random.choice(foo))
```

For cryptographically secure random choices (e.g., for generating a passphrase from a wordlist), use `secrets.choice()`:

```
Python
1
2
3
4
import secrets

foo = ['battery', 'correct', 'horse', 'staple']

print(secrets.choice(foo))
```

`secrets` is new in Python 3.6. On older versions of Python you can use the `random.SystemRandom` class:

```
Python
1
2
import random
```

```
secure_random = random.SystemRandom()
print(secure_random.choice(foo))
```

3
4
5

88) How do I get the number of elements in a list in Python?

How do I get the number of elements in the list items?

```
items = ["apple", "orange", "banana"]
```

Answer#

The len() function can be used with several different types in Python – both built-in types and library types.

For example:

```
len([1, 2, 3])
3
```

89) How to remove an element from a list by index in Python?

Answer#

Use del and specify the index of the element you want to delete:

```
Python
>>> a = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> del a[-1]
>>> a
```

1
2
3

```
[0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
#Also supports slices:
```

```
>>> del a[2:4]
```

```
>>> a
```

```
[0, 1, 4, 5, 6, 7, 8, 9]
```

90) How do I count the occurrences of a list item in Python?

Answer#

If you only want a single item's count, use the count method:

```
[1, 2, 3, 4, 1, 4, 1].count(1)
```

```
3
```

Important: this is very slow if you are counting multiple different items

Each count call goes over the entire list of n elements. Calling count in a loop n times means $n * n$ total checks, which can be catastrophic for performance.

If you want to count multiple items, use Counter, which only does n total checks.

Python

```
>>> from collections import Counter
```

```
>>> z = ['blue', 'red', 'blue', 'yellow', 'blue', 'red']
```

```
>>> Counter(z)
```

```
Counter({'blue': 3, 'red': 2, 'yellow': 1})
```

91) Why is it string.join(list) instead of list.join(string)?

Explain the below code:

Why not

```
["Hello", "world"].join("-")
```

Why this:

```
"-".join(["Hello", "world"])
```

Is there a specific reason it is like this?

Answer#

It's because any iterable can be joined (e.g, list, tuple, dict, set), but its contents and the "joiner" must be strings.

For example:

```
''.join(['welcome', 'to', 'python', 'programming'])  
''.join(('welcome', 'to', 'python', 'programming'))
```

```
'welcome_to_python_programming'
```

Using something other than strings will raise the following error:

```
TypeError: sequence item 0: expected str instance, int found
```

Python Arrays Coding Interview Questions

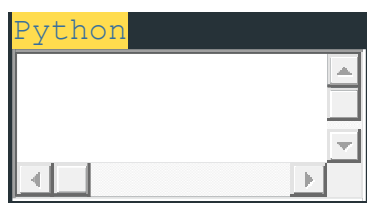
92) Is arr.__len__() the preferred way to get the length of an array in Python?

In Python, is the following the only way to get the number of elements?

arr.__len__()

If so, why the strange syntax?

Answer#

A small window titled "Python" with a white background and a grey border, containing a text editor with a few lines of code.

```
Python
#For Arrays:

my_list = [1,2,3,4,5]
len(my_list)
# 5

#The same works for tuples:

my_tuple = (1,2,3,4,5)
len(my_tuple)
# 5

#And strings, which are really just arrays of characters:

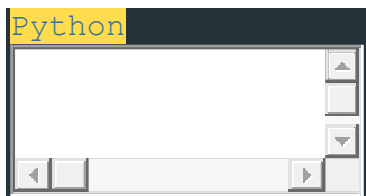
my_string = 'hello world'
len(my_string)
```

It was intentionally done this way so that lists, tuples and other container types or iterables didn't all need to explicitly implement a public `.length()` method, instead you can just check the `len()` of anything that implements the 'magic' `__len__()` method.

93) How to check if multiple strings exist in another string in Python?

Answer#

You can use any:



```
1 a_string = "A string is more than its parts!"
2
3 matches = ["more", "wholesome", "milk"]
4
5 if any(x in a_string for x in matches):
6
#Similarly to check if all the strings from the list are found, use all instead of any.
```

94) Python list vs. array – when to use?

If you are creating a 1d array, you can implement it as a list, or else use the 'array' module in the standard library. I have always used lists for 1d arrays.

What is the reason or circumstance where I would want to use the array module instead?

Is it for performance and memory optimization, or am I missing something obvious?

Answer#

Basically, Python lists are very flexible and can hold completely heterogeneous, arbitrary data, and they can be appended to very efficiently, in amortized constant time. If you need to shrink and grow your list time-efficiently and without hassle, they are the way to go.

But they use a lot more space than C arrays, in part because each item in the list requires the construction of an individual Python object, even for data that could be represented with simple C types (e.g. float or uint64_t).

The `array.array` type, on the other hand, is just a thin wrapper on C arrays. It can hold only homogeneous data (that is to say, all of the same type) and so it uses only `sizeof(one object) * length` bytes of memory. Mostly, you should use it when you need to expose a C array to an extension or a system call (for example, `ioctl` or `fctnl`).

`array.array` is also a reasonable way to represent a mutable string in Python 2.x (`array('B', bytes)`). However, Python 2.6+ and 3.x offer a mutable byte string as `bytearray`.

However, if you want to do math on a homogeneous array of numeric data, then you're much better off using NumPy, which can automatically vectorize operations on complex multi-dimensional arrays.

To make a long story short: `array.array` is useful when you need a homogeneous C array of data for reasons other than doing math.

95) How do I print the full NumPy array, without truncation?

Answer#



```
Python
#Use numpy.set_printoptions:
```

```
import sys
import numpy
numpy.set_printoptions(threshold=sys.maxsize)
```

96) How do I dump a NumPy array into a csv file in a human-readable format?

Answer#

```
Python
#numpy.savetxt saves an array to a text file.

import numpy
a = numpy.asarray([[1,2,3], [4,5,6], [7,8,9]])
numpy.savetxt("foo.csv", a, delimiter=",")
```

97) How do I access the ith column of a NumPy multidimensional array?

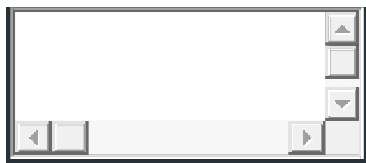
Given:

```
test = numpy.array([[1, 2], [3, 4], [5, 6]])
```

test[i] gives the ith row (e.g. [1, 2]). How do I access the ith column? (e.g. [1, 3, 5]). Also, would this be an expensive operation?

Answer#

```
Python
```



```
#To access column 0:

>>> test[:, 0]

array([1, 3, 5])

To access row 0:

>>> test[0, :]

array([1, 2])
```

98) Is there a NumPy function to return the first index of something in an array?

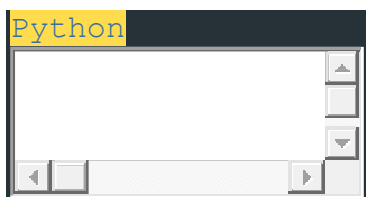
Answer#

Yes, given an array, array, and a value, item to search for, you can use `np.where` as:

```
itemindex = numpy.where(array == item)
```

The result is a tuple with first all the row indices, then all the column indices.

For example, if an array is two dimensions and it contained your item at two locations then



```
Python

array[itemindex[0][0]][itemindex[1][0]]
```

```
#would be equal to your item and so would be:
```

```
array[itemindex[0][1]][itemindex[1][1]]
```

99) What are the advantages of NumPy over regular Python lists?

Answer#

NumPy's arrays are more compact than Python lists — a list of lists as you describe, in Python, would take at least 20 MB or so, while a NumPy 3D array with single-precision floats in the cells would fit in 4 MB. Access in reading and writing items is also faster with NumPy.

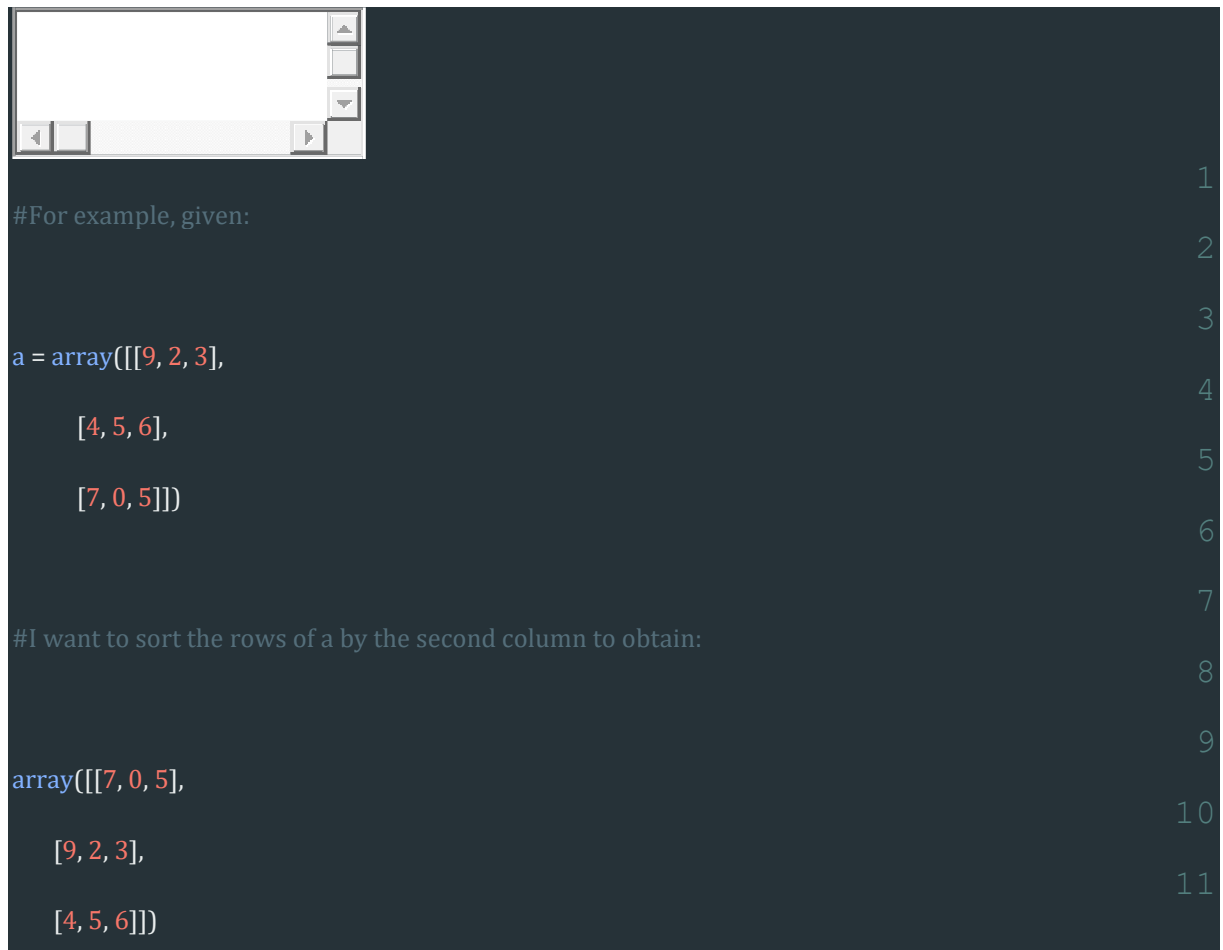
Maybe you don't care that much for just a million cells, but you definitely would for a billion cells — neither approach would fit in a 32-bit architecture, but with 64-bit builds NumPy would get away with 4 GB or so, Python alone would need at least about 12 GB (lots of pointers which double in size) — a much costlier piece of hardware!

The difference is mostly due to "indirectness" — a Python list is an array of pointers to Python objects, at least 4 bytes per pointer plus 16 bytes for even the smallest Python object (4 for type pointer, 4 for reference count, 4 for value — and the memory allocators rounds up to 16).

A NumPy array is an array of uniform values — single-precision numbers takes 4 bytes each, double-precision ones, 8 bytes. Less flexible, but you pay substantially for the flexibility of standard Python lists!

100) Explain sorting arrays in NumPy by column?

Python



```
1
2
3
4
5
6
7
8
9
10
11
#For example, given:

a = array([[9, 2, 3],
          [4, 5, 6],
          [7, 0, 5]])

#I want to sort the rows of a by the second column to obtain:

array([[7, 0, 5],
       [9, 2, 3],
       [4, 5, 6]])
```

Answer#

To sort by the second column of a:

```
a[a[:, 1].argsort()]
```

101) How to get numpy array dimensions?

How do I get the dimensions of an array? For instance, this is 2×2:

```
a = np.array([[1,2],[3,4]])
```

Answer#

Use .shape to obtain a tuple of array dimensions:

```
a.shape
(2, 2)
```

102) What is the difference between np.array() and np.asarray()?

What is the difference between NumPy's np.array and np.asarray? When should I use one rather than the other? They seem to generate identical output.

Answer#

The definition of asarray is:

```
def asarray(a, dtype=None, order=None):  
    return array(a, dtype, copy=False, order=order)
```

So it is like array, except it has fewer options, and copy=False. array has copy=True by default.

The main difference is that array (by default) will make a copy of the object, while asarray will not unless necessary.

103) How do I concatenate two one-dimensional arrays in NumPy?

A screenshot of a Python IDE with a dark background. The title bar says 'Python'. The code editor contains the following code:

```
import numpy as np  
a = np.array([1, 2, 3])  
b = np.array([4, 5])  
np.concatenate(a, b)
```

Line numbers 1, 2, 3, and 4 are visible on the right side of the editor.

Answer#

Use:

```
np.concatenate([a, b])
```

The arrays you want to concatenate need to be passed in as a sequence, not as separate arguments.

From the NumPy documentation:

```
numpy.concatenate((a1, a2, ...), axis=0)
```

Join a sequence of arrays together.

It was trying to interpret your `b` as the `axis` parameter, which is why it complained it couldn't convert it into a scalar.

104) What is the difference between `ndarray` and `array` in NumPy?

Where is their implementation in the NumPy source code?

Answer#

`numpy.array` is just a convenience function to create an `ndarray`; it is not a class itself.

You can also create an array using `numpy.ndarray`, but it is not the recommended way. From the docstring of `numpy.ndarray`:

Arrays should be constructed using `array`, `zeros` or `empty` ... The parameters given here refer to a low-level method (`ndarray(...)`) for instantiating an array.

105) How does `np.einsum` work?

Given arrays `A` and `B`, their matrix multiplication followed by transpose is computed using `(A @ B).T`, or equivalently, using:

```
np.einsum("ij, jk -> ki", A, B)
```

Answer#

What does einsum do?

Imagine that we have two multi-dimensional arrays, A and B. Now let's suppose we want to...

multiply A with B in a particular way to create new array of products; and then maybe sum this new array along particular axes; and then maybe transpose the axes of the new array in a particular order.

There's a good chance that einsum will help us do this faster and more memory-efficiently than combinations of the NumPy functions like multiply, sum and transpose will allow.

106) How can I remove some specific elements from a numpy array? Say I have

```
import numpy as np
```

```
a = np.array([1,2,3,4,5,6,7,8,9])
```

I then want to remove 3,4,7 from a. All I know is the index of the values (index=[2,3,6]).

Answer#

Use `numpy.delete()` – returns a new array with sub-arrays along an axis deleted

```
numpy.delete(a, index)
```

For your specific question:

A screenshot of a Python IDE with a dark background. The word 'Python' is in a yellow box at the top left. Below it is a text editor window with a white background. The code 'import numpy as np' is written in the editor. To the right of the editor are several small icons: a triangle pointing up, a square, a triangle pointing down, and a square. At the bottom right of the IDE, there are two small numbers, '1' and '2', stacked vertically.

```
import numpy as np
```

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
index = [2, 3, 6]

new_a = np.delete(a, index)

print(new_a) #Prints `[1, 2, 5, 6, 8, 9]`
```

Note that numpy.delete() returns a new array since array scalars are immutable, similar to strings in Python, so each time a change is made to it, a new object is created.

Python Coding Interview Questions And Answers For Experienced

107) How do you debug a Python program?

Answer) By using this command we can debug a [python](#) program



The screenshot shows a terminal window with a dark background. At the top, the word "Python" is written in yellow. Below it, there is a white rectangular area with a black border, containing a small icon of a document with a magnifying glass. At the bottom of the terminal, a red prompt character "\$" is followed by the command "python -m pdb python-script.py" in green text.

```
$ python -m pdb python-script.py
```

108) What is <Yield> Keyword in Python?

A) The <yield> keyword in Python can turn any function into a generator. Yields work like a standard return keyword.

But it'll always return a generator object. Also, a function can have multiple calls to the <yield> keyword.

Example:

```
Python
def testgen(index):
    weekdays = ['sun','mon','tue','wed','thu','fri','sat']
    yield weekdays[index]
    yield weekdays[index+1]
day = testgen(0)
print next(day), next(day)
```

Output: sun mon

109) How to convert a list into a string?

A) When we want to convert a list into a string, we can use the `<".join()>` method which joins all the elements into one and returns as a string.

Example:

```
Python
weekdays = ['sun','mon','tue','wed','thu','fri','sat']
listAsString = ''.join(weekdays)
print(listAsString)
```

110) How to convert a list into a tuple?

A) By using Python `<tuple()>` function we can convert a list into a tuple. But we can't change the list after turning it into tuple, because it becomes immutable.

Example:

```
Python
weekdays = ['sun','mon','tue','wed','thu','fri','sat']
listAsTuple = tuple(weekdays)
print(listAsTuple)
```

output: ('sun', 'mon', 'tue', 'wed', 'thu', 'fri', 'sat')

111) How to convert a list into a set?

A) User can convert list into set by using <set()> function.

Example:

```
Python
weekdays = ['sun','mon','tue','wed','thu','fri','sat','sun','tue']
listAsSet = set(weekdays)
print(listAsSet)
```

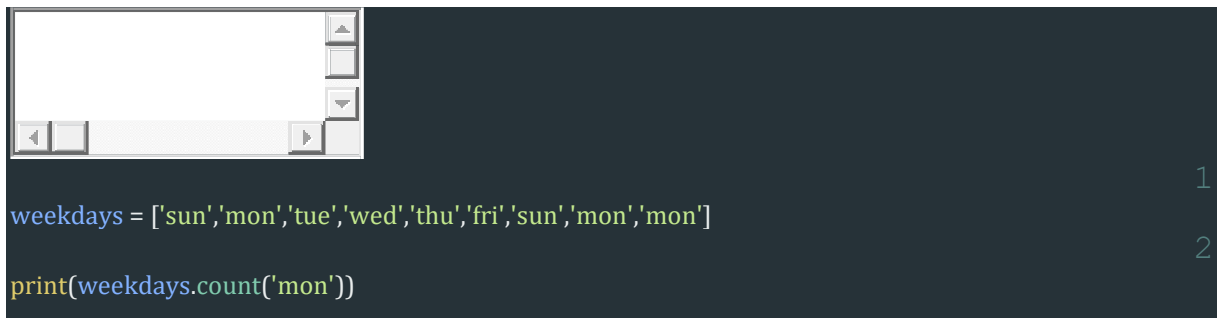
output: set(['wed', 'sun', 'thu', 'tue', 'mon', 'fri', 'sat'])

112) How to count the occurrences of a particular element in the list?

A) In Python list, we can count the occurrences of an individual element by using a <count()> function.

Example # 1:

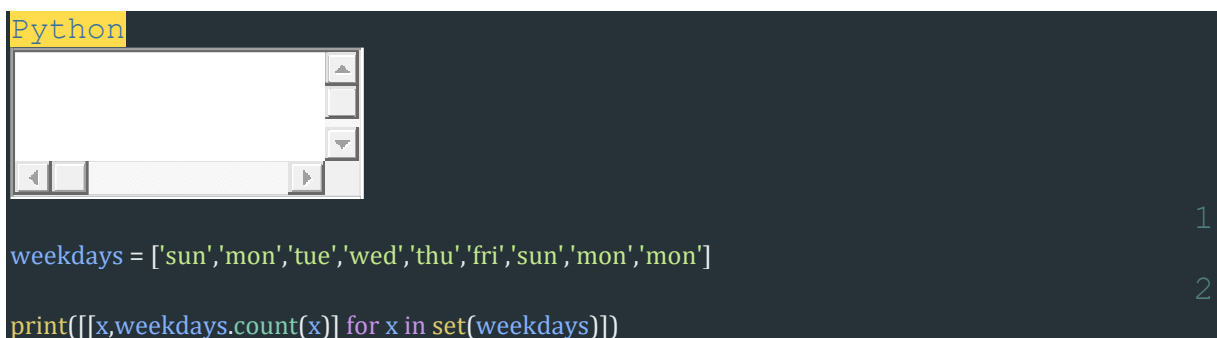
```
Python
```



```
1 weekdays = ['sun','mon','tue','wed','thu','fri','sun','mon','mon']
2 print weekdays.count('mon')
```

Output: 3

Example # 2:



```
Python
1 weekdays = ['sun','mon','tue','wed','thu','fri','sun','mon','mon']
2 print([[x,weekdays.count(x)] for x in set(weekdays)])
```

output: [['wed', 1], ['sun', 2], ['thu', 1], ['tue', 1], ['mon', 3], ['fri', 1]]

113) What is NumPy array?

A) NumPy arrays are more flexible than lists in Python. By using NumPy arrays reading and writing items is faster and more efficient.

114) How can you create Empty NumPy Array In Python?

A) We can create Empty NumPy Array in two ways in Python,

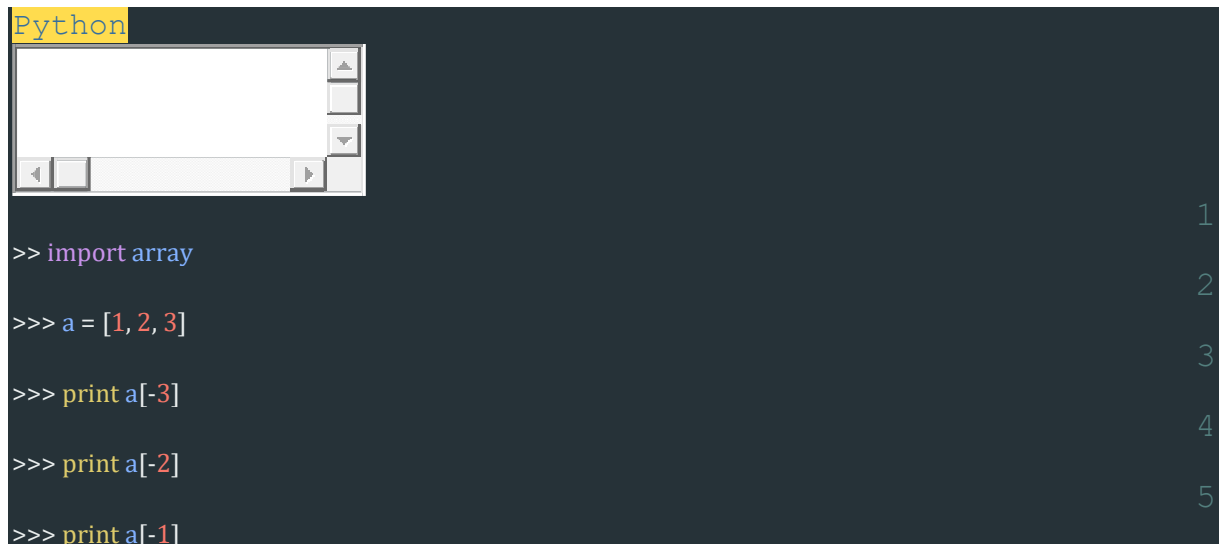
1) import numpy
numpy.array([])

2) numpy.empty(shape=(0,0))

115) What is a negative index in Python?

A) Python has a special feature like a negative index in Arrays and Lists. Positive index reads the elements from the starting of an array or list but in the negative index, Python reads elements from the end of an array or list.

116) What is the output of the below code?

A screenshot of a Python IDE with a dark background. The title bar says 'Python'. The code editor contains the following code:

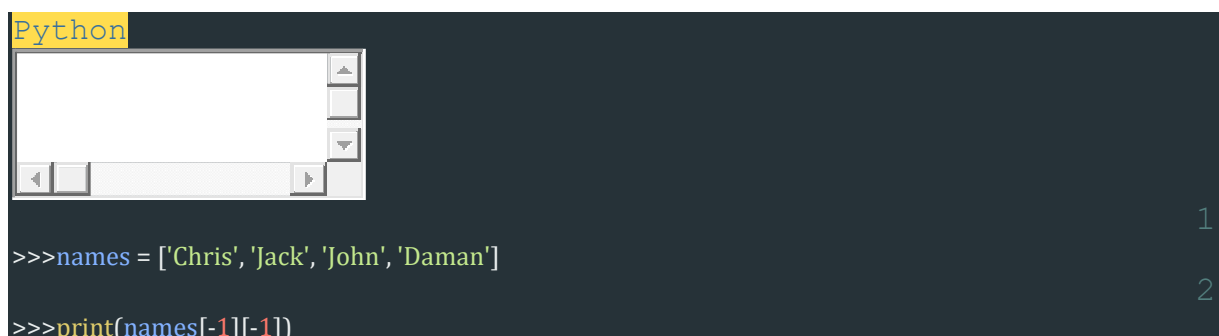
```
>> import array  
  
>>> a = [1, 2, 3]  
  
>>> print a[-3]  
  
>>> print a[-2]  
  
>>> print a[-1]
```

Line numbers 1 through 5 are visible on the right side of the editor.

A) The output is: 3, 2, 1

Advanced Python Coding Interview Questions

117) What is the output of the below program?

A screenshot of a Python IDE with a dark background. The title bar says 'Python'. The code editor contains the following code:

```
>>> names = ['Chris', 'Jack', 'John', 'Daman']  
  
>>> print(names[-1][-1])
```

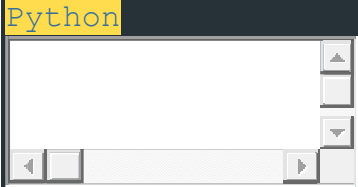
Line numbers 1 and 2 are visible on the right side of the editor.

A) The output is: n

118) What is Enumerate() Function in Python?

A) The Python enumerate() function adds a counter to an iterable object. enumerate() function can accept sequential indexes starting from zero.

Python Enumerate Example:



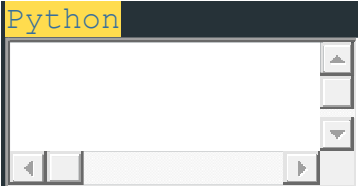
```
1 subjects = ('Python', 'Interview', 'Questions')
2 for i, subject in enumerate(subjects):
3     print(i, subject)
```

Output:

```
0 Python
1 Interview
2 Questions
```

119) What is data type SET in Python and how to work with it?

A) The Python data type "set" is a kind of collection. It has been part of Python since version 2.4. A set contains an unordered collection of unique and immutable objects.



```
1 # *** Create a set with strings and perform a search in set
2
3 objects = {"python", "coding", "tips", "for", "beginners"}
```

```
4
# Print set.
5
print(objects)
6
print(len(objects))
7
8
# Use of "in" keyword.
9
if "tips" in objects:
10
    print("These are the best Python coding tips.")
11
12
# Use of "not in" keyword.
13
if "Java tips" not in objects:
14
    print("These are the best Python coding tips not Java tips.")
15
16
# ** Output
17
{'python', 'coding', 'tips', 'for', 'beginners'}
18
5
19
These are the best Python coding tips.
20
These are the best Python coding tips not Java tips.
21
22
# *** Lets initialize an empty set
23
items = set()
24
25
# Add three strings.
26
items.add("Python")
27
items.add("coding")
28
items.add("tips")
29
```

```
print(items)
```

```
# ** Output
```

```
{'Python', 'coding', 'tips'}
```

120) How do you Concatenate Strings in Python?

A) We can use '+' to concatenate strings.

Python Concatenating Example:

Python

```
# See how to use '+' to concatenate strings.
```

```
>>> print('Python' + ' Interview' + ' Questions')
```

```
# Output:
```

Python Interview Questions

121) How to generate random numbers in Python?

A) We can generate random numbers using different functions in Python. They are:

#1. random() – This command returns a floating point number, between 0 and 1.

#2. uniform(X, Y) – It returns a floating point number between the values given as X and Y.

#3. randint(X, Y) – This command returns a random integer between the values given as X and Y.

122) How to print the sum of the numbers starting from 1 to 100?

A) We can print sum of the numbers starting from 1 to 100 using this code:

A screenshot of a Python code editor with a dark background. The word "Python" is in a yellow tab at the top left. The code "print sum(range(1,101))" is written in a light blue font. On the right side, there is a vertical line of numbers 1 through 6, with the number 1 being highlighted in green.

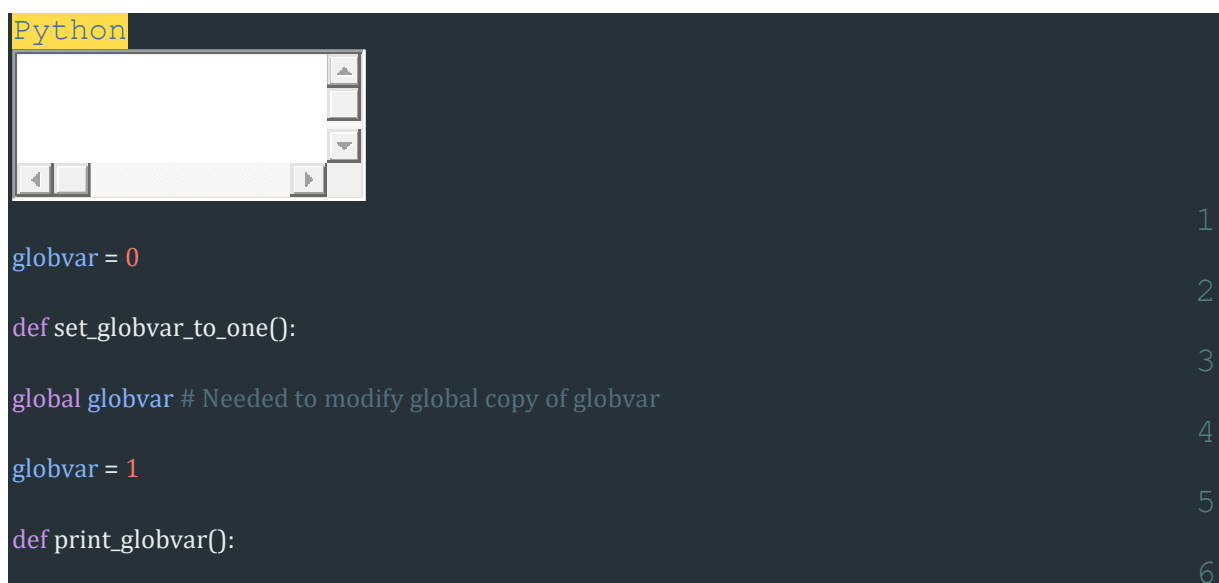
```
Python  
print sum(range(1,101))
```

In Python the range function does not include the end given. Here it will exclude 101.

Sum function print sum of the elements of range function, i.e 1 to 100.

123) How do you set a global variable inside a function?

A) Yes, we can use a global variable in other functions by declaring it as global in each function that assigns to it:

A screenshot of a Python code editor with a dark background. The word "Python" is in a yellow tab at the top left. The code is written in a light blue font. On the right side, there is a vertical line of numbers 1 through 6, with the number 1 being highlighted in green.

```
Python  
globvar = 0  
  
def set_globvar_to_one():  
    global globvar # Needed to modify global copy of globvar  
    globvar = 1  
  
def print_globvar():
```

```
print globvar # No need for global declaration to read value of globvar
```

7

```
set_globvar_to_one()
```

8

```
print_globvar() # Prints 1
```

124) What is the output of the program?

Python

```
names1 = ['Amir', 'Bear', 'Charlton', 'Daman']
```

1

```
names2 = names1
```

2

```
names3 = names1[:]
```

3

```
names2[0] = 'Alice'
```

4

5

```
names3[1] = 'Bob'
```

6

7

```
sum = 0
```

8

```
for ls in (names1, names2, names3):
```

9

```
if ls[0] == 'Alice':
```

10

11

```
sum += 1
```

12

```
if ls[1] == 'Bob':
```

13

```
sum += 10
```

14

```
print sum
```

15

A) 12

125) What is the output, Suppose list1 is [1, 3, 2], What is list1 * 2?

A) [1, 3, 2, 1, 3, 2]

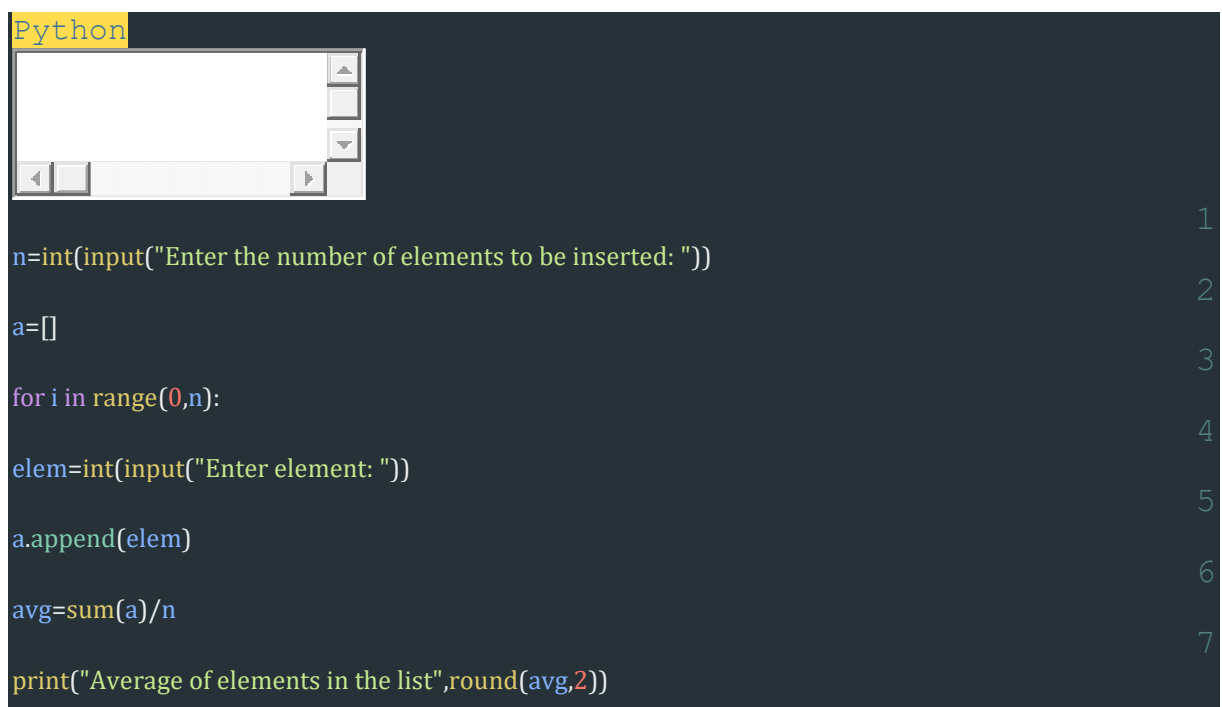
126) What is the output when we execute `list("hello")`?

A) ['h', 'e', 'l', 'l', 'o']

Python Coding Interview Questions And Answers For Experienced

127) Can you write a program to find the average of numbers in a list in Python?

A) Python Program to Calculate Average of Numbers:

A screenshot of a Python IDE with a dark background. The title bar shows 'Python'. The code is as follows:

```
1 n=int(input("Enter the number of elements to be inserted: "))
2
3 a=[]
4
5 for i in range(0,n):
6     elem=int(input("Enter element: "))
7     a.append(elem)
8
9 avg=sum(a)/n
10
11 print("Average of elements in the list",round(avg,2))
```

Output:

Enter the number of elements to be inserted: 3

Enter element: 23

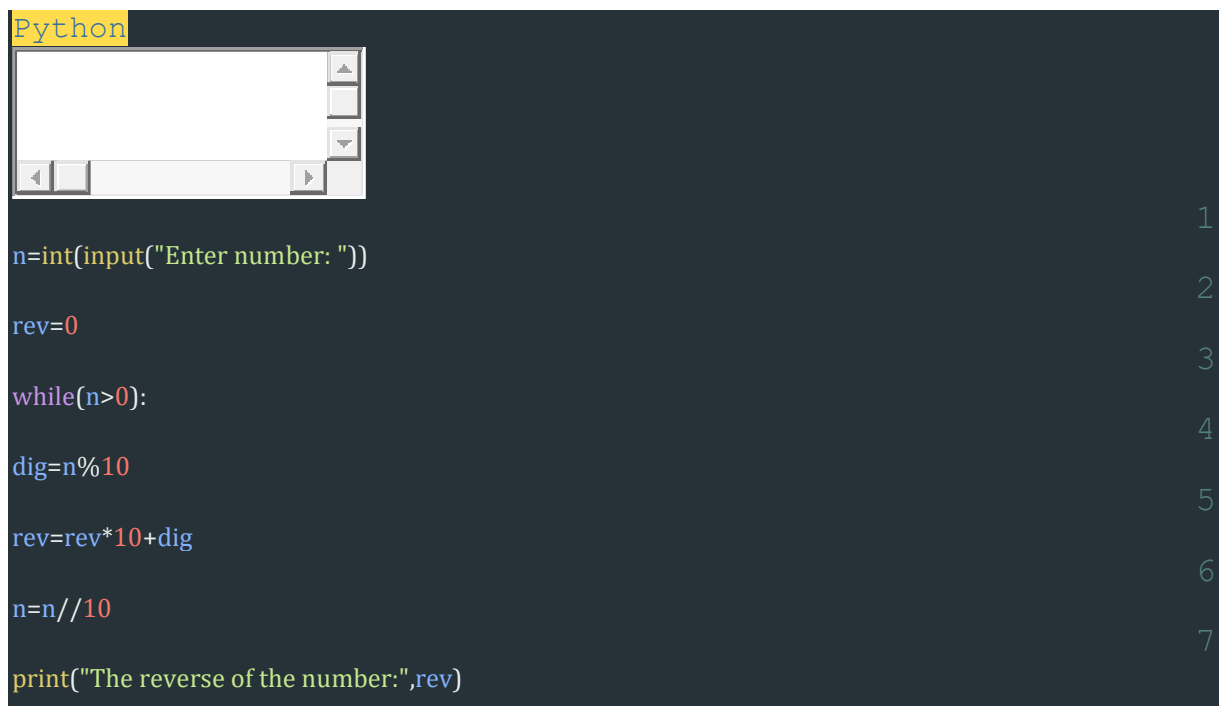
Enter element: 45

Enter element: 56

Average of elements in the list 41.33

128) Write a program to reverse a number in Python?

A) Python Program to Reverse a Number:

A screenshot of a Python IDE. The top part shows a file explorer with a folder named 'Python'. Below it is a code editor with a dark background. The code is as follows:

```
n=int(input("Enter number: "))  
  
rev=0  
  
while(n>0):  
    dig=n%10  
    rev=rev*10+dig  
    n=n//10  
  
print("The reverse of the number:",rev)
```

On the right side of the code editor, there are line numbers 1 through 7.

Output:

Enter number: 143

The reverse of the number: 341

129) Write a program to find the sum of the digits of a number in Python?

A) Python Program to Find Sum of the Digits of a Number

A screenshot of a Python IDE. The top part shows a file explorer with a folder named 'Python'. Below it is a code editor with a dark background. The code is as follows:

```
n=int(input("Enter number: "))  
  
sum=0  
  
while(n>0):  
    dig=n%10  
    sum=sum+dig  
    n=n//10  
  
print("The sum of the digits of the number:",sum)
```

```

n=int(input("Enter a number:"))
tot=0
while(n>0):
    dig=n%10
    tot=tot+dig
    n=n//10
print("The total sum of digits is:",tot)

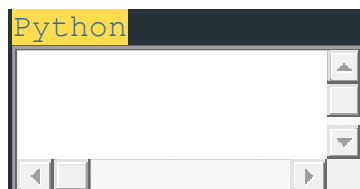
```

Output:

Enter a number:1928
The total sum of digits is: 20

130) Write a Python Program to Check if a Number is a Palindrome or not?

A) Python Program to Check if a Number is a Palindrome or Not:



```

n=int(input("Enter number:"))
temp=n
rev=0
while(n>0):
    dig=n%10
    rev=rev*10+dig
    n=n//10
if(temp==rev):

```

```

print("The number is a palindrome!")
else:
print("The number isn't a palindrome!")

```

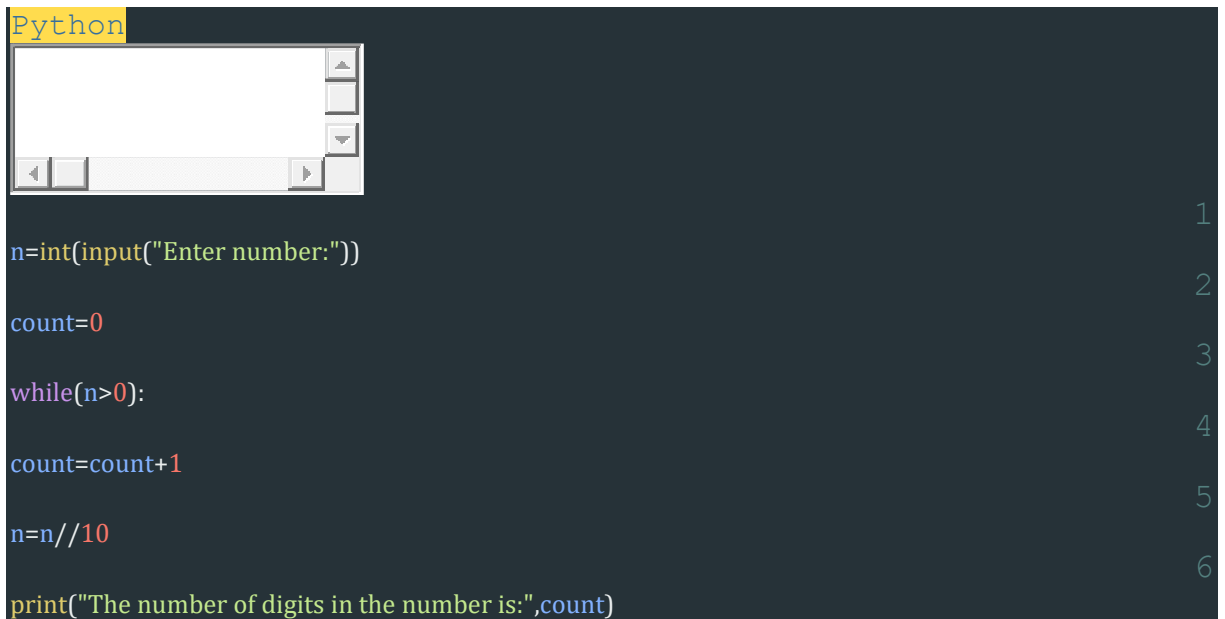
Output:

Enter number:151

The number is a palindrome!

131) Write a Python Program to Count the Number of Digits in a Number?

A) Python Program to Count the Number of Digits in a Number:



```

Python
n=int(input("Enter number:"))
count=0
while(n>0):
count=count+1
n=n//10
print("The number of digits in the number is:",count)

```

Output:

Enter number:14325

The number of digits in the number is: 5

132) Write a Python Program to Print Table of a Given Number?

A) Python Program to Print Table of a Given Number:

```
Python
n=int(input("Enter the number to print the tables for:"))
for i in range(1,11):
    print(n,"x",i,"=",n*i)
```

Output:

Enter the number to print the tables for:7

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

133) Write a Python Program to Check if a Number is a Prime Number?

A) Python Program to Check if a Number is a Prime Number:

```
Python
a=int(input("Enter number: "))
k=0
for i in range(2,a//2+1):
    if(a%i==0):
```

```

k=k+1

if(k<=0):

print("Number is prime")

else:

print("Number isn't prime")

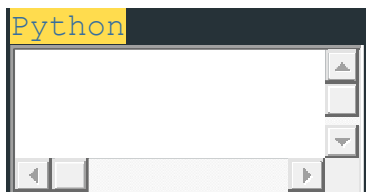
```

Output:

Enter number: 7
Number is prime

134) Write a Python Program to Check if a Number is an Armstrong Number?

A) Python Program to Check if a Number is an Armstrong Number:



```

n=int(input("Enter any number: "))

a=list(map(int,str(n)))

b=list(map(lambda x:x**3,a))

if(sum(b)==n):

print("The number is an armstrong number. ")

else:

print("The number isn't an arsmtrong number. ")

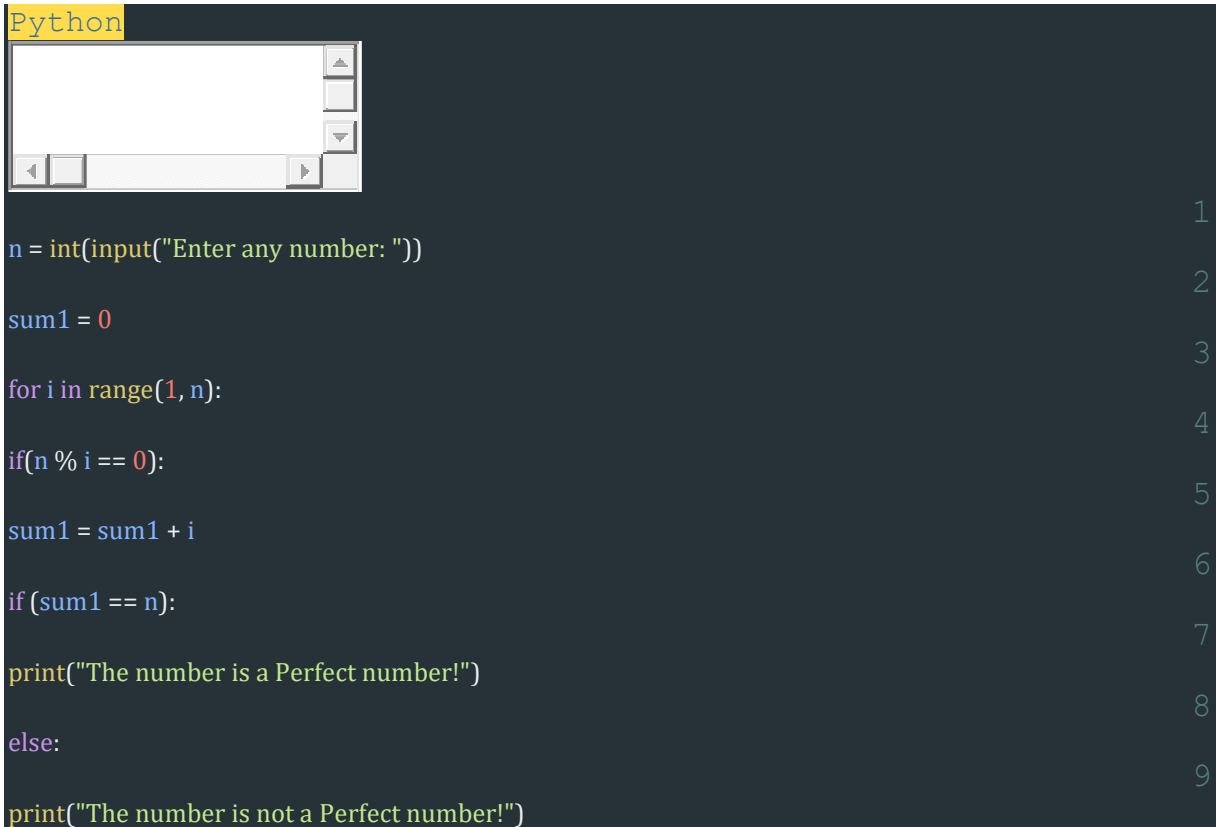
```

Output:

Enter any number: 371
The number is an armstrong number.

135) Write a Python Program to Check if a Number is a Perfect Number?

A) Python Program to Check if a Number is a Perfect Number:

A screenshot of a Python IDE window. The window has a title bar with the word "Python" in a yellow tab. Below the title bar is a text area with a dark background and light-colored text. The text contains a Python program that checks if a number is a perfect number. The program uses input, range, if, and print statements. The code is as follows:

```
n = int(input("Enter any number: "))
sum1 = 0
for i in range(1, n):
    if(n % i == 0):
        sum1 = sum1 + i
if (sum1 == n):
    print("The number is a Perfect number!")
else:
    print("The number is not a Perfect number!")
```

The code is numbered 1 through 9 on the right side of the window.

Output:

Enter any number: 6
The number is a Perfect number!

Python Developer Interview Questions And Answers

136) Write a Python Program to Check if a Number is a Strong Number?

A) Python Program to Check if a Number is a Strong Number:

```
Python
sum1=0
num=int(input("Enter a number:"))
temp=num
while(num):
    i=1
    f=1
    r=num%10
    while(i<=r):
        f=f*i
        i=i+1
    sum1=sum1+f
    num=num//10
    if(sum1==temp):
        print("The number is a strong number")
    else:
        print("The number is not a strong number")
```

Output:

Enter a number:145
The number is a strong number.

137) Write a Python Program to Find the Second Largest Number in a List?

A) Python Program to Find the Second Largest Number in a List:

```
Python
a=[]
n=int(input("Enter number of elements:"))
for i in range(1,n+1):
    b=int(input("Enter element:"))
    a.append(b)
a.sort()
print("Second largest element is:",a[n-2])
```

Output:

Enter number of elements:4
Enter element:23
Enter element:56
Enter element:39
Enter element:11
Second largest element is: 39

138) Write a Python Program to Swap the First and Last Value of a List?

A) Python Program to Swap the First and Last Value of a List:

```
Python
a=[]
n= int(input("Enter the number of elements in list:"))
for x in range(0,n):
```

```

element=int(input("Enter element" + str(x+1) + ":"))
a.append(element)
temp=a[0]
a[0]=a[n-1]
a[n-1]=temp
print("New list is:")
print(a)

```

Output:

Enter the number of elements in list:4
Enter element1:23
Enter element2:45
Enter element3:67
Enter element4:89
New list is:
[89, 45, 67, 23]

139) Write a Python Program to Check if a String is a Palindrome or Not?

A) Python Program to Check if a String is a Palindrome or Not:

```

Python
string=raw_input("Enter string:")
if(string==string[::-1]):
print("The string is a palindrome")
else:
print("The string isn't a palindrome")

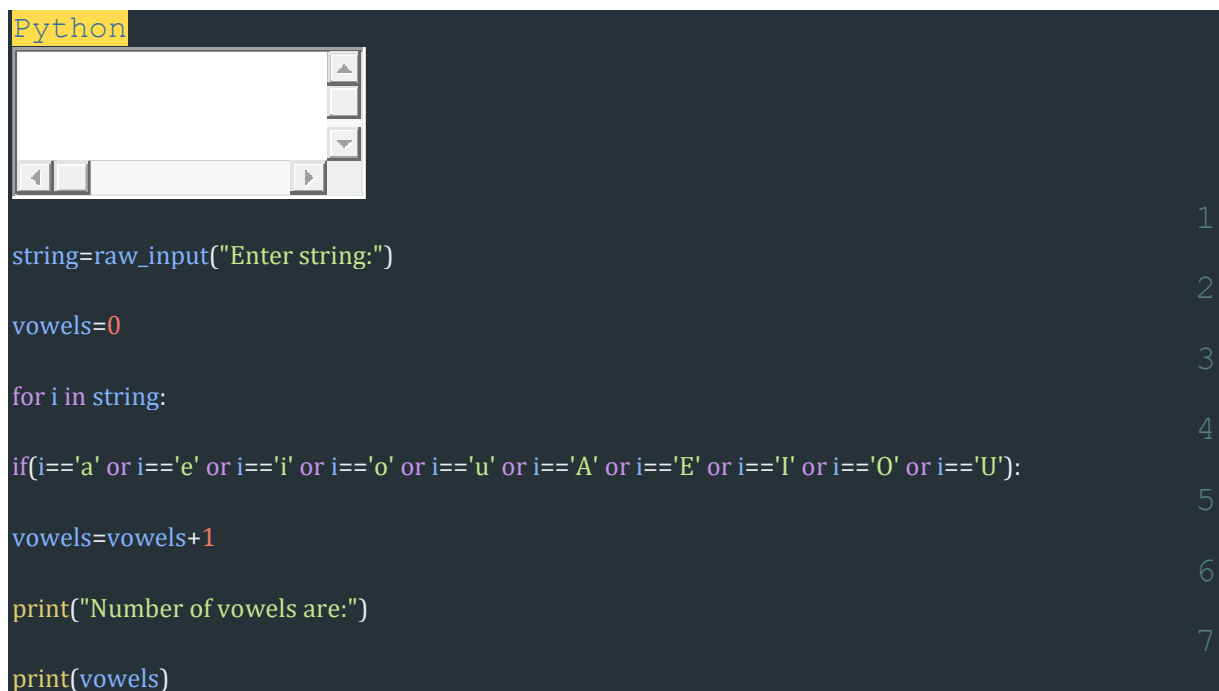
```

Output:

Enter string: malayalam
The string is a palindrome

140) Write a Python Program to Count the Number of Vowels in a String?

A) Python Program to Count the Number of Vowels in a String:

A screenshot of a Python IDE with a dark background. The word 'Python' is in the top-left corner. A small window with a scrollbar is open. The code is as follows:

```
string=raw_input("Enter string:")  
vowels=0  
for i in string:  
    if(i=='a' or i=='e' or i=='i' or i=='o' or i=='u' or i=='A' or i=='E' or i=='I' or i=='O' or i=='U'):  
        vowels=vowels+1  
print("Number of vowels are:")  
print(vowels)
```

Line numbers 1 through 7 are visible on the right side of the code editor.

Output:

Enter string: Hello world
Number of vowels are: 3

141) Write a Python Program to Check Common Letters in Two Input Strings?

A) Python Program to Check Common Letters in Two Input Strings:

A screenshot of a Python IDE with a dark background. The word 'Python' is in the top-left corner. A small window with a scrollbar is open. The code is not visible in this image.

```
s1=raw_input("Enter first string:")
s2=raw_input("Enter second string:")
a=list(set(s1)&set(s2))
print("The common letters are:")
for i in a:
print(i)
```

Output:

```
Enter first string:Hello
Enter second string:How are you
The common letters are:
H
e
o
```

Must Read Python Interview Questions

- [165+ Python Interview Questions & Answers](#)

Other Python Tutorials

- [What is Python?](#)
- [Python Advantages](#)
- [Python For Beginners](#)
- [Python For Machine Learning](#)
- [Machine Learning For Beginners](#)
- [130+ Python Projects With Source Code On GitHub](#)

Related Interview Questions From Codingcompiler

1. [CSS3 Interview Questions](#)
2. [Linux Administrator Interview Questions](#)
3. [SQL Interview Questions](#)
4. [Hibernate Interview Questions](#)
5. [Kubernetes Interview Questions](#)
6. [Kibana Interview Questions](#)
7. [Nagios Interview Questions](#)

8. [Jenkins Interview Questions](#)
9. [Chef Interview Questions](#)
10. [Puppet Interview Questions](#)
11. [RPA Interview Questions And Answers](#)
12. [Android Interview Questions](#)
13. [Mulesoft Interview Questions](#)
14. [JSON Interview Questions](#)
15. [PeopleSoft HRMS Interview Questions](#)
16. [PeopleSoft Functional Interview Questions](#)
17. [PeopleTools Interview Questions](#)
18. [Peoplesoft Technical Interview Questions](#)
19. [199 Peoplesoft Interview Questions](#)
20. [200 Blue Prism Interview Questions](#)
21. [Visualforce Interview Questions](#)
22. [Salesforce Interview Questions](#)
23. [300 SSIS Interview Questions](#)
24. [PHP Interview Questions And Answers](#)
25. [Alteryx Interview Questions](#)
26. [AWS Cloud Support Interview Questions](#)
27. [Google Kubernetes Engine Interview Questions](#)
28. [AWS Devops Interview Questions](#)
29. [Apigee Interview Questions](#)
30. [Actimize Interview Questions](#)

Python Coding Interview Questions

Related FAQs

How To Practice Python Coding For Interviews?

Python programming is a widely used high-level, general-purpose, interpreted, and dynamic programming language for modern application development. If you are a novice programmer, you must attend any good Python training course and practice Python coding every day with different concepts. Participate in forums, coding challenges, hackathons, bug bounty programs, and learn python coding interview questions here.

Is Python Programming Allowed In Coding Inte

